

Министерство образования Республики Беларусь
Учреждение образования
Белорусский государственный университет информатики и
радиоэлектроники
Кафедра электронных вычислительных машин

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту на тему
«Разработка микро-ЭВМ на ПЛИС»

Студент: Власов Р.Е.

Руководитель: _____

Минск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Разработка общей структуры микро-ЭВМ	6
1.1 Функциональный состав микро-ЭВМ.....	6
1.2 Разработка системы команд	10
1.3 Описание взаимодействия всех блоков микро-ЭВМ при выполнении команд программы	13
2 Разработка основных устройств микро-ЭВМ	18
2.1 Запоминающее устройство.....	18
2.2 Устройство управления	24
2.3 Регистры общего назначения.....	32
2.4 Блок общих операций	33
2.4.1 Блок нулевой итерации.....	39
2.4.2 Блоки арбитража к общим шинам	40
2.4.3 Блок операции HLT (останов микро-ЭВМ).....	41
2.4.4 Блок операции JMP (безусловный переход)	41
2.4.5 Блок операции M->R (чтение из RAM в РОН)	42
2.4.6 Блок операции R->M (запись из РОН в RAM).....	43
2.5 Арифметико-логические устройства	44
2.5.1 Блоки арбитража к общим шинам.....	46
2.5.2 Блок регистров арифметико-логического устройства	46
2.5.3 Блок операции OR (побитовое «ИЛИ»).....	47
2.5.4 Блок операции NOR (побитовое «ИЛИ-НЕ»)	47
2.5.5 Блок операции SRA (арифметический сдвиг вправо)	48
2.5.6 Блок операции INCS (инкремент по флагу S).....	48
2.5.7 Блок формирования регистра флагов.....	49
2.6 Стековое запоминающее устройство	50
2.6.1 Блоки арбитража к общим шинам	51
2.6.2 Блок схемы стека.....	52
2.7 КЭШ-память	53
2.7.1 Блок данных КЭШа.....	53
2.7.2 Второстепенные блоки КЭШа	56
2.7.3 Блок управления КЭШа.....	57
2.8 Контроллер прямого доступа к памяти.....	61
2.9 Предсказатель переходов	63
2.10 Организация арбитража.....	64
3 Функциональное моделирование	66
3.1 Функциональное моделирование блока памяти	66

3.2 Функциональное моделирование декодера команд	68
3.3 Функциональное моделирование счётчика фаз команд	70
3.4 Функциональное моделирование блока IP и IR.....	72
3.5 Функциональное моделирование стекового устройства	74
3.6 Функциональное моделирование регистров общего назначения	77
3.7 Функциональное моделирование АЛУ.....	79
3.8 Функциональное моделирование кэш-памяти.....	81
3.9 Функциональное моделирование предсказателя переходов	83
3.10 Функциональное моделирование КПП	85
3.11 Функциональное моделирование арбитража шин.....	87
3.12 Функциональное моделирование полной микро-ЭВМ.....	89
ЗАКЛЮЧЕНИЕ	94
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	95
ПРИЛОЖЕНИЕ А (обязательное) Схема комбинированная структурная блока памяти.....	96
ПРИЛОЖЕНИЕ Б (обязательное) Схема комбинированная структурная блока декодера команд	97
ПРИЛОЖЕНИЕ В (обязательное) Схема комбинированная структурная блока счётчика фаз команд	98
ПРИЛОЖЕНИЕ Г (обязательное) Схема комбинированная структурная блока указателя команд и регистра кода команд.....	99
ПРИЛОЖЕНИЕ Д (обязательное) Схема комбинированная структурная блока стекового устройства	100
ПРИЛОЖЕНИЕ Е (обязательное) Схема комбинированная структурная блока регистров общего назначения	101
ПРИЛОЖЕНИЕ Ж (обязательное) Схема комбинированная структурная блока арифметико-логического устройства	102
ПРИЛОЖЕНИЕ И (обязательное) Схема комбинированная структурная блока кэш-памяти.....	103
ПРИЛОЖЕНИЕ К (обязательное) Схема комбинированная структурная блока предсказателя переходов	104
ПРИЛОЖЕНИЕ Л (обязательное) Схема комбинированная структурная блока контроллера прямого доступа к памяти.....	105
ПРИЛОЖЕНИЕ М (обязательное) Схема комбинированная структурная блока арбитража шин.....	106
ПРИЛОЖЕНИЕ Н (обязательное) Схема комбинированная структурная блока микро-ЭВМ	107
ПРИЛОЖЕНИЕ П (обязательное) Листинг проекта	108

ВВЕДЕНИЕ

Целью курсового проекта является разработка микро-ЭВМ на ПЛИС в соответствии с индивидуальным техническим заданием по дисциплине «Структурная и функциональная организация ЭВМ». Введение фиксирует основные исходные параметры системы, от которых далее зависят структура команд, состав функциональных блоков, набор управляющих сигналов и перечень временных диаграмм.

Разрабатываемая микро-ЭВМ имеет следующие основные магистрали:

- шина данных разрядностью 16 бит;
- адресная шина разрядностью 16 бит;
- управляющая шина, разрядность которой определяется набором сигналов чтения и записи памяти, разрешения регистров, выбора источников внутренней шины, управления АЛУ, стеком, кэшем, КПДП, арбитром и предсказателем переходов.

Архитектура микро-ЭВМ принята гарвардской: ПЗУ команд и ОЗУ данных имеют отдельные тракты обращения. Это позволяет выборке команды не конфликтовать с обычным доступом к данным, однако обращения процессора и КПДП к ОЗУ всё равно требуют арбитража. Команды имеют фиксированный двухсловный формат: первое слово содержит код операции и номер регистра, второе слово используется как прямой адрес операнда, адрес перехода или резервное поле.

Тип адресации в проекте прямо-регистровый. Операнд или приёмник результата задаётся номером одного из 12 регистров общего назначения R0-R11, а адрес ячейки памяти располагается во втором слове команды. Для команд JMP и JZ второе слово содержит адрес следующей команды в пространстве ПЗУ.

Арифметико-логическое устройство должно выполнять операции INCS, OR, NOR и SRA. Операция INCS использует значение флага S, операции OR и NOR выполняют побитовую логическую обработку, а SRA реализует арифметический сдвиг вправо с сохранением знакового бита. Результаты операций сопровождаются формированием флагов Z, S, C и O.

Кэш-память данных организуется как четырёхканальная множественно-ассоциативная структура с вытеснением строки по наибольшей давности хранения. Синхронизация с основной памятью выполняется по схеме сквозной записи: при записи слово согласуется и с кэшем, и с ОЗУ, поэтому последующее чтение получает актуальное значение.

Арбитр шин реализуется как централизованный параллельный блок. Его назначение - исключить одновременное управление общей магистралью данных со стороны процессорной части и контроллера прямого доступа к памяти. Блок предсказания переходов выполняется отдельно от устройства управления и использует схему А4 с индексированием по сочетанию младших разрядов счётчика команд и регистра глобальной истории GHR.

Стек реализуется отдельным блоком глубиной 7 слов по 16 бит с направлением роста вниз. Контроллер прямого доступа к памяти передаёт в ОЗУ блок объёмом 6 байт, что при словной организации памяти соответствует трём 16-разрядным словам, начиная с адреса 000Ah.

Разрабатываемая микро-ЭВМ является многотактовой, а не конвейерной. Поэтому выполнение команды рассматривается как последовательность фаз выборки, декодирования, обращения к операндам, исполнения и записи результата. Такой подход упрощает проверку временных диаграмм и позволяет отдельно показать работу ПЗУ, ОЗУ, АЛУ, стека, кэша, КПДП, арбитра и предсказателя переходов.

Разработка и проверка проекта выполняются в среде проектирования цифровых устройств с использованием структурного описания и функционального моделирования. Итогом работы должна стать согласованная структура микро-ЭВМ, набор приложений с графическими схемами и листингами, а также результаты проверки, подтверждающие корректность работы отдельных блоков и всей системы.

1 РАЗРАБОТКА ОБЩЕЙ СТРУКТУРЫ МИКРО-ЭВМ

В данном разделе определяется архитектурная основа проектируемой системы, фиксируется набор её функциональных блоков, разрабатывается система команд и описывается взаимодействие всех узлов при исполнении программы. Именно этот раздел задаёт ту логическую модель, по которой далее будут проектироваться отдельные схемы, временные диаграммы и результаты моделирования.

1.1 Функциональный состав микро-ЭВМ

Разрабатываемая микро-ЭВМ построена по Гарвардской архитектуре, то есть память команд и память данных разделены. ПЗУ используется только для хранения и выборки команд программы, а ОЗУ предназначено для хранения операндов, промежуточных результатов и данных, передаваемых контроллером прямого доступа к памяти. Такое разделение упрощает организацию выборки команд и уменьшает число конфликтов между обращениями к программе и данным [8].

Разрядность шины данных составляет 16 бит, разрядность адресной шины также составляет 16 бит. Команда имеет фиксированный формат и состоит из двух 16-разрядных слов: первое слово содержит код операции и номер регистра, второе слово используется как адресная часть. Управление работой всех блоков выполняется централизованно устройством управления, которое формирует управляющие сигналы для регистров, АЛУ, стека, кэша, ОЗУ, КППД и предсказателя переходов [10].

1.1.1 Блок запоминающих устройств

Блок запоминающих устройств включает ПЗУ команд, ОЗУ данных, кэш-память данных и схемы согласования обращений к памяти. ПЗУ команд хранит машинную программу и выдаёт два последовательных слова команды, которые загружаются в регистры IR0 и IR1. ОЗУ данных используется для хранения операндов, результатов выполнения операций и области обмена с контроллером прямого доступа к памяти.

Кэш-память данных применяется для ускорения обращений процессора к ОЗУ. Она имеет множественно-ассоциативную организацию 4-way, 16 наборов и строку размером одно слово. В кэше хранятся теги, признаки действительности строк и счётчики давности, используемые при выборе

строки для замещения. Запись реализована по принципу write-through, поэтому изменённые данные передаются в основную память [1].

Доступ к ОЗУ согласуется через арбитр шины. Это необходимо, так как к памяти данных могут обращаться как процессорное ядро через кэш, так и контроллер прямого доступа к памяти. Арбитр предотвращает одновременную запись или чтение несколькими источниками и выдаёт разрешение активному устройству.

1.1.2 Устройство управления

Устройство управления является центральным блоком микро-ЭВМ. Оно выполняет выборку команды из ПЗУ, декодирует код операции, определяет номер регистра и формирует последовательность управляющих сигналов для остальных функциональных блоков.

В состав устройства управления входят регистры текущей команды IR0 и IR1, декодер кода операции и номера регистра, фазовый автомат выполнения команды, схема выбора следующего значения указателя команд IP и формирователь управляющих сигналов. Фазовый автомат задаёт последовательность тактов выполнения команды и при необходимости вводит дополнительный такт ожидания Tw, например при обращении к памяти.

Устройство управления также взаимодействует с предсказателем переходов. При выполнении команд перехода оно получает предполагаемый адрес следующей команды, а после фактического выполнения перехода обновляет состояние предсказателя.

1.1.3 Блок регистров общего назначения

Блок регистров общего назначения предназначен для хранения операндов и промежуточных результатов вычислений. В микро-ЭВМ реализовано 12 регистров R0-R11 разрядностью 16 бит.

Регистровый файл имеет два порта чтения и один порт записи. Это позволяет одновременно считывать операнды для выполнения операции и записывать результат после завершения такта исполнения. Регистры общего назначения адресуются полем REG_NUM команды и используются пользователем для хранения данных программы.

1.1.4 Арифметико-логическое устройство

Арифметико-логическое устройство выполняет операции OR, NOR, SRA и INCS. Операции OR и NOR используют регистровый операнд и слово, считанное из памяти данных. Операция SRA выполняет арифметический сдвиг содержимого выбранного регистра вправо. Операция INCS увеличивает значение регистра на значение флага S [4].

После выполнения арифметико-логической операции формируются флаги Z, S, C и O. Эти флаги сохраняются в регистре FR и используются устройством управления при выполнении условных переходов и при анализе результата вычислений.

1.1.5 Блок стека

Стек реализован отдельным функциональным блоком глубиной 7 слов по 16 бит. Он организован по принципу LIFO: последним записанное значение считывается первым. Направление роста стека принято вниз.

Для работы со стеком используются команды PUSH и POP. Команда PUSH записывает содержимое выбранного регистра общего назначения в стек, а команда POP извлекает слово из стека и помещает его в выбранный регистр. В составе блока предусмотрены указатель вершины SP и схемы контроля пустого и полного состояния, предотвращающие некорректные операции со стеком.

1.1.6 Контроллер прямого доступа к памяти

Контроллер прямого доступа к памяти предназначен для обмена данными с ОЗУ без участия арифметико-логического тракта процессора. В данной микро-ЭВМ КПДП выполняет передачу объёмом 6 байт, то есть 3 слова по 16 бит, начиная с адреса 000Ah.

КПДП взаимодействует с ОЗУ через арбитр шины. При необходимости обмена он формирует запрос доступа к памяти, после чего арбитр выдаёт разрешение. Такой подход позволяет согласовать работу процессора и КПДП и исключить конфликт одновременного обращения к ОЗУ [2].

1.1.7 Предсказатель переходов

Для ускорения выполнения команд передачи управления используется предсказатель переходов по схеме А4. Он формирует предполагаемый адрес следующей команды до завершения фактического анализа условия перехода. В состав предсказателя входят таблица истории переходов, буфер целевых адресов и регистр глобальной истории GHR. После выполнения условного перехода устройство управления передаёт в предсказатель фактический результат, на основании которого обновляются внутренние таблицы. Если предсказание оказалось неверным, устройство управления корректирует значение IP и продолжает выборку команды с правильного адреса.

1.1.8 Специальные регистры микро-ЭВМ

Помимо регистров общего назначения, микро-ЭВМ содержит специальные регистры, которые используются аппаратной логикой и не адресуются командами напрямую.

К специальным регистрам относятся:

- IP — 16-разрядный указатель команды, задающий адрес первого слова текущей или следующей команды в ПЗУ;
- IR0 и IR1 — регистры двух слов команды; IR0 хранит код операции и номер регистра, IR1 хранит адресную часть;
- FR — регистр флагов Z, S, C и O;
- SP — указатель вершины стека;
- GHR — регистр глобальной истории переходов;
- служебные регистры КПДП — текущий адрес передачи, счётчик оставшихся байт и буфер данных;
- служебные регистры кэша — теги, признаки действительности строк и счётчики давности хранения.

Специальные регистры не предназначены для хранения пользовательских данных. Их содержимое формируется аппаратно и зависит от текущей фазы выполнения команды. Для хранения данных программы используются регистры общего назначения R0-R11 и ОЗУ данных.

Таким образом, функциональный состав микро-ЭВМ включает устройство управления, блоки памяти, регистровый файл, арифметико-логическое устройство, стек, кэш-память, КПДП, арбитр шины и предсказатель переходов. Все эти блоки образуют многотактную вычислительную систему, в которой устройство управления задаёт

последовательность выполнения команд, а исполнительные и служебные блоки выполняют отдельные операции обмена, вычисления и управления потоком программы.

1.2 Разработка системы команд

Система команд обеспечивает обмен с памятью, работу АЛУ, стек и передачу управления

Количество реализованных команд - 11:

- HLT - останов выполнения программы;
- MOV adr, Rn - чтение слова из памяти данных в регистр общего назначения;
- MOV Rn, adr - запись слова из регистра общего назначения в память данных;
- OR Rn, adr - побитовое ИЛИ регистра и слова памяти;
- NOR Rn, adr - инверсия результата побитового ИЛИ;
- SRA Rn - арифметический сдвиг регистра вправо;
- INCS Rn - инкремент регистра на значение флага S;
- PUSH Rn - занесение регистра в стек;
- POP Rn - извлечение слова из стека в регистр;
- JMP adr - безусловная загрузка адреса в IP;
- JZ adr - условный переход при установленном флаге Z.

Для кодирования одиннадцати команд достаточно четырёх бит, однако в проекте поле кода операции принято восьмибитным. Такое решение согласуется с 16-разрядной организацией слова команды, упрощает аппаратное декодирование и оставляет резерв кодов для возможного расширения системы команд. Неиспользуемые коды рассматриваются как зарезервированные и в аппаратной реализации переводят процессор в безопасное состояние останова [5].

Команда имеет фиксированную длину два 16-разрядных слова. Первое слово содержит код операции и номер регистра, второе слово используется как адрес операнда в ОЗУ, адрес перехода в ПЗУ или резервное поле для команд, которым адрес не требуется. Фиксированный формат выбран для того, чтобы фазовый автомат всегда выполнял одинаковую процедуру выборки IR0 и IR1.

Положение полей команды фиксировано:

- OP CODE[7..0] - код операции, всегда размещается в разрядах IR0(15..8);
- REG_NUM[3..0] - номер регистра общего назначения, размещается в разрядах IR0(7..4);

- X[3..0] - служебное поле IR0(3..0), не влияющее на выполнение текущего набора команд;
- ADDR[15..0] - адресная часть IR1(15..0), используемая как адрес ОЗУ, адрес перехода в ПЗУ или резервное поле;
- RESERVED - коды и поля, оставленные для дальнейшего расширения системы команд.

Поле REG_NUM имеет разрядность 4 бита, поэтому теоретически позволяет адресовать шестнадцать регистров. В проектируемой микро-ЭВМ фактически используются регистры R0-R11, а коды 12-15 в данной реализации не используются и считаются зарезервированными.

Адресное поле второго слова интерпретируется в зависимости от команды. Для MOV, OR и NOR оно задаёт адрес в ОЗУ данных; для JMP и JZ - адрес команды в ПЗУ; для HLT, SRA, INCS, PUSH и POP второе слово выбирается из ПЗУ вместе с командой, но при исполнении не используется.

Кодировка КОП принята следующей:

- 00h - HLT;
- 01h - MOV adr, Rn;
- 02h - MOV Rn, adr;
- 03h - OR Rn, adr;
- 04h - NOR Rn, adr;
- 05h - SRA Rn;
- 06h - INCS Rn;
- 07h - PUSH Rn;
- 08h - POP Rn;
- 09h - JMP adr;
- 0Ah - JZ adr.

Любой другой код операции считается зарезервированным. В аппаратной модели такой код не передаётся на выполнение произвольному блоку, а приводит процессор к безопасному останову, что упрощает отладку и защищает модель от случайного выполнения неопределённой команды.

Принятая система команд делится на четыре группы: команды обмена с памятью, арифметико-логические команды, стековые команды и команды передачи управления. Команда JZ включена для содержательной проверки предсказателя переходов: без условного перехода предсказатель невозможно показать в работе, так как безусловный JMP не требует анализа результата вычисления.

Таблица 1.1 – Структура системы команд микро-ЭВМ

№	Команда	Назначение	Формат IR0	Формат IR1	КОП, hex
1	HLT	останов выполнения программы	OPCODE=00h, REG_NUM=X, X=X	не используется	00h
2	MOV adr, Rn	чтение слова из ОЗУ в регистр Rn	OPCODE=01h, REG_NUM=n, X=X	адрес ОЗУ ADDR[1 5..0]	01h
3	MOV Rn, adr	запись слова из Rn в ОЗУ	OPCODE=02h, REG_NUM=n, X=X	адрес ОЗУ ADDR[1 5..0]	02h
4	OR Rn, adr	побитовое ИЛИ Rn и слова ОЗУ	OPCODE=03h, REG_NUM=n, X=X	адрес ОЗУ ADDR[1 5..0]	03h
5	NOR Rn, adr	инверсия результата OR	OPCODE=04h, REG_NUM=n, X=X	адрес ОЗУ ADDR[1 5..0]	04h
6	SRA Rn	арифметический сдвиг Rn вправо	OPCODE=05h, REG_NUM=n, X=X	не используется	05h
7	INCS Rn	инкремент Rn на значение флага S	OPCODE=06h, REG_NUM=n, X=X	не используется	06h
8	PUSH Rn	занесение Rn в стек	OPCODE=07h, REG_NUM=n, X=X	не используется	07h
9	POP Rn	извлечение слова из стека в Rn	OPCODE=08h, REG_NUM=n, X=X	не используется	08h
10	JMP adr	безусловная загрузка адреса в IP	OPCODE=09h, REG_NUM=X, X=X	адрес перехода AD DR[15..0]	09h
11	JZ adr	переход по адресу при флаге Z	OPCODE=0Ah, REG_NUM=X, X=X	адрес перехода AD DR[15..0]	0Ah

В таблице обозначение n соответствует номеру регистра общего назначения R0-R11.

Обозначение X указывает на поле, значение которого не влияет на выполнение команды. Для команд HLT, SRA, INCS, PUSH и POP второе слово команды считывается из ПЗУ в рамках единого формата, однако при исполнении не используется.

Для наглядности можно привести примеры кодирования команд. Команда MOV 000Ah, R3, выполняющая чтение слова из ячейки ОЗУ с адресом 000Ah в регистр R3, кодируется следующим образом: первое слово IR0 = 0130h, второе слово IR1 = 000Ah. Команда JZ 0012h, выполняющая переход по адресу 0012h при установленном флаге Z, кодируется как IR0 = 0A00h, IR1 = 0012h.

Таким образом, принятый формат команды является регулярным: устройство управления всегда выбирает из ПЗУ два 16-разрядных слова, после чего использует только те поля, которые требуются конкретной операции. Это упрощает фазовый автомат выборки и декодирования команд.

1.3 Описание взаимодействия всех блоков микро-ЭВМ при выполнении команд программы

1.3.1 Роль устройства управления

Главным блоком в иерархии микроЭВМ является устройство управления. Именно оно выполняет выборку двухсловной команды из ПЗУ, фиксирует слова команды в регистрах IR0 и IR1, декодирует код операции и номер регистра, а затем формирует последовательность управляющих сигналов для остальных узлов. Исполнительные блоки не работают независимо от устройства управления: они получают разрешение на чтение, запись или изменение состояния только в те такты, которые заданы фазовым автоматом T0...T5 и дополнительным тактом ожидания Tw. Общая структурная схема микро-ЭВМ приведена в приложении Н; она связывает процессорное ядро, память, кэш, КППД, арбитр шин и предсказатель переходов в единую систему.

Непосредственно извне для работы микроЭВМ задаются тактовый сигнал CLK, сигнал сброса RESET, содержимое ПЗУ команд и начальные данные ОЗУ. Дальнейшее поведение системы определяется последовательностью команд программы. После декодирования команды устройство управления выб

ирует маршрут данных: обращение к памяти через кэш, выполнение операции в АЛУ, обмен со стеком, изменение указателя команд или останов.

Выборка команд из ПЗУ не конкурирует с обращениями к ОЗУ данных, поскольку в проекте используется Гарвардская архитектура: память команд и память данных разделены. Это позволяет устройству управления выполнять выборку очередной команды независимо от операций чтения и записи данных.

1.3.2 Исполнительные и служебные блоки

К исполнительным и служебным блокам, взаимодействующим при выполнении программы, относятся:

- блок общих операций: HLT, JMP, JZ, MOV adr, Rn и MOV Rn, adr;
- арифметико-логический блок: OR, NOR, SRA и INCS;
- блок стека: PUSH, POP, память стека и указатель SP;
- блок регистров общего назначения R0-R11 и регистр флагов FR;
- подсистема памяти: кэш данных, ОЗУ, ПЗУ команд и мультиплексоры адресов/данных;
- служебные подсистемы: арбитр общей шины, КПДП и предсказатель переходов A4.

Для команд обмена с памятью активизируются кэш, ОЗУ, регистровый файл и арбитр шины.

Для арифметикологических команд основными исполнительными блоками являются РОН, АЛУ и регистр флагов. Для стековых команд дополнительно используется стековая память и указатель SP. Для команд передачи управления задействуются IP, устройство управления и предсказатель переходов. Общий фрагмент структурной схемы проектируемой микро-ЭВМ приведён на рисунке 1.1. На нём показаны основные блоки верхнего уровня: процессорное ядро, кэш, ОЗУ, ПЗУ, КПДП, арбитр шины и предсказатель переходов. По этому фрагменту удобно проследить, что обмен данными внутри системы идёт не напрямую между всеми блоками, а через ограниченный набор общих линий и управляющих разрешений.

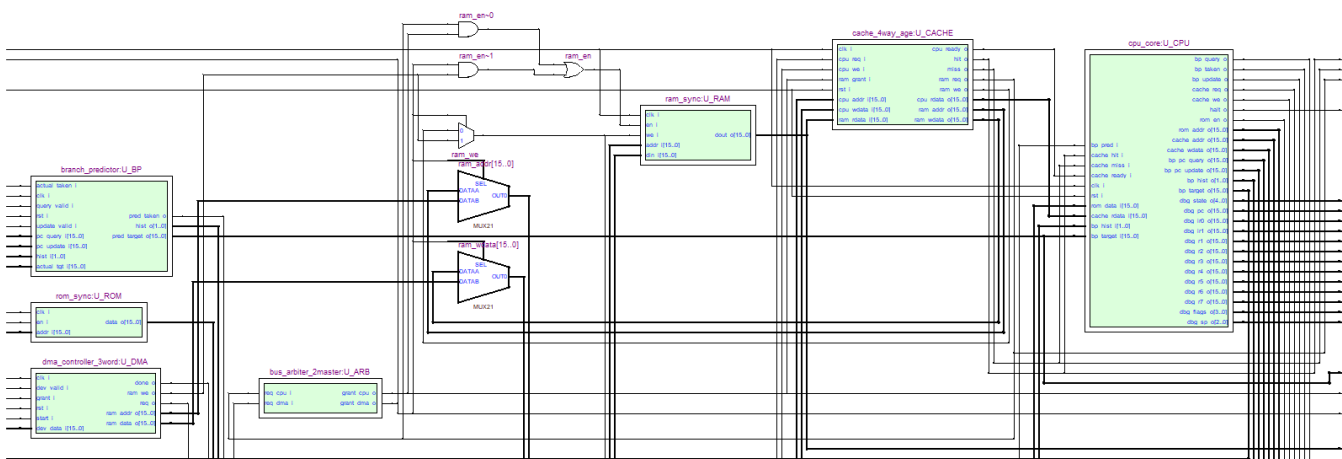


Рисунок 1.1 - Символическое представление взаимодействия устройства управления, исполнительных и служебных блоков микро-ЭВМ

1.3.3 Взаимодействие с памятью и кэшем

При выполнении команд обмена с памятью процессорное ядро формирует адрес, направление обмена и слово данных. Запрос поступает в кэш данных. При попадании кэш возвращает требуемое слово без обращения к ОЗУ, при промахе или записи формируется обращение к синхронной памяти данных.

Кэш имеет организацию 4-way, 16 наборов, строку размером одно слово и использует сквозную запись write-through. Для выбора строки при замещении применяются счётчики давности хранения. Такая организация позволяет ускорить повторные обращения к данным и одновременно сохраняет согласованность с основной памятью.

Так как к ОЗУ может обращаться не только процессор, но и КПДП, доступ к общей памяти согласуется арбитром шины. В данной реализации арбитр выдаёт разрешающие сигналы grant_cpu и grant_dma и тем самым не допускает одновременной записи разными источниками в одну и ту же память. При одновременных запросах приоритет получает КПДП, так как его обмен с памятью должен выполняться без участия арифметико-логического тракта процессора.

Фрагмент схемы подсистемы памяти приведён на рисунке 1.2. Здесь показаны кэш, ОЗУ, КПДП, арбитр и мультиплексоры выбора адреса и данных. Такая организация особенно важна для Гарвардской архитектуры проекта: ПЗУ команд используется только для выборки программы, а ОЗУ данных обслуживает операции чтения/записи, кэширование и передачу данных через КПДП.

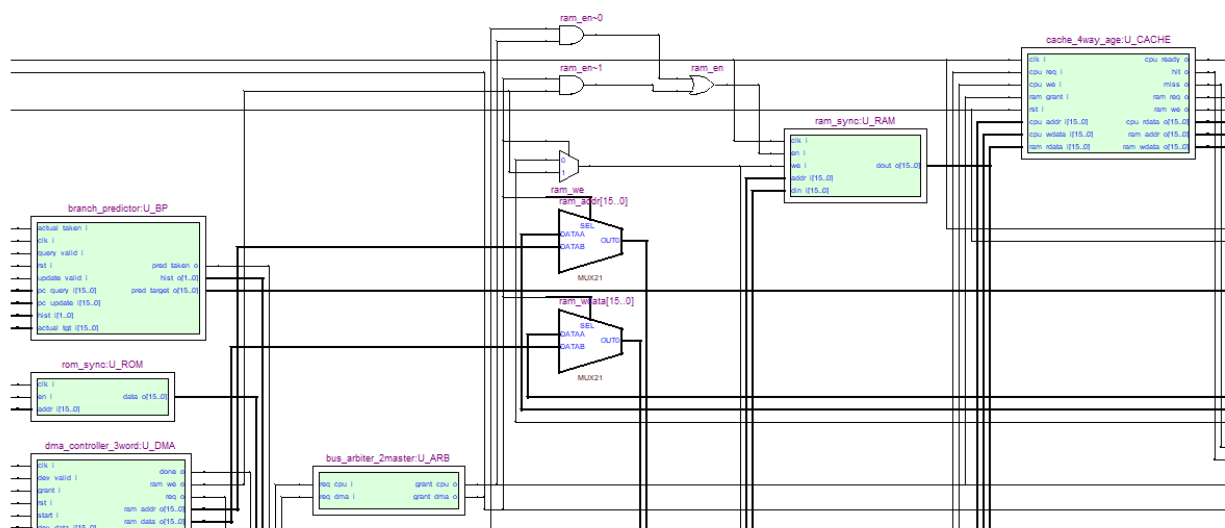


Рисунок 1.2 – Символическое представление взаимодействия кэша данных, ОЗУ, КПДП и арбитра шины

1.3.4 Взаимодействие с регистрами, АЛУ и стеком

Для арифметико-логических и стековых команд устройство управления разрешает чтение выбранного регистра Rn, передаёт операнды на входы АЛУ или в стековый блок и фиксирует результат в регистровом файле. При выполнении OR и NOR второй операнд считывается из памяти данных через кэш, поэтому эти команды задействуют одновременно регистровый файл, АЛУ и подсистему памяти.

Команда SRA использует только регистровый операнд и выполняет арифметический сдвиг выбранного регистра вправо. Команда INCS увеличивает значение регистра на величину, определяемую флагом S. После выполнения арифметико-логических операций обновляются флаги Z, S, C и O, которые затем могут использоваться устройством управления при выполнении условного перехода.

Команды PUSH и POP работают через стек глубиной 7 слов. При PUSH устройство управления разрешает чтение регистра и запись слова в текущую вершину стека, после чего изменяется указатель SP. При POP выполняется обратная операция: слово считывается из стека, записывается в выбранный регистр общего назначения, затем указатель вершины корректируется. Состояния пустого и полного стека контролируются аппаратно, чтобы исключить некорректное изменение памяти стека.

1.3.5 Предсказание переходов и завершение выполнения команды

Команды передачи управления меняют содержимое IP. Для безусловного перехода JMP в IP загружается адрес из второго слова команды. Для условного перехода JZ дополнительно проверяется флаг Z. Если условие выполнено, в IP загружается адрес перехода; если условие не выполнено, выполнение продолжается со следующей команды.

Предсказатель переходов A4 получает текущий адрес команды и историю переходов, выдаёт предполагаемый адрес следующей команды и обновляет свои таблицы после фактического выполнения перехода. Если предсказание оказалось неверным, устройство управления сбрасывает ошибочно выбранное направление и продолжает выборку с правильного адреса.

Таким образом, взаимодействие всех блоков организовано централизованно: устройство управления задаёт текущую фазу команды и активные разрешающие сигналы, исполнительные блоки выполняют только назначенную им часть операции, а арбитр и кэш ограничивают доступ к общей памяти. Такая структура уменьшает вероятность конфликтов на шинах и делает выполнение команд предсказуемым при функциональном моделировании и проверке схемы.

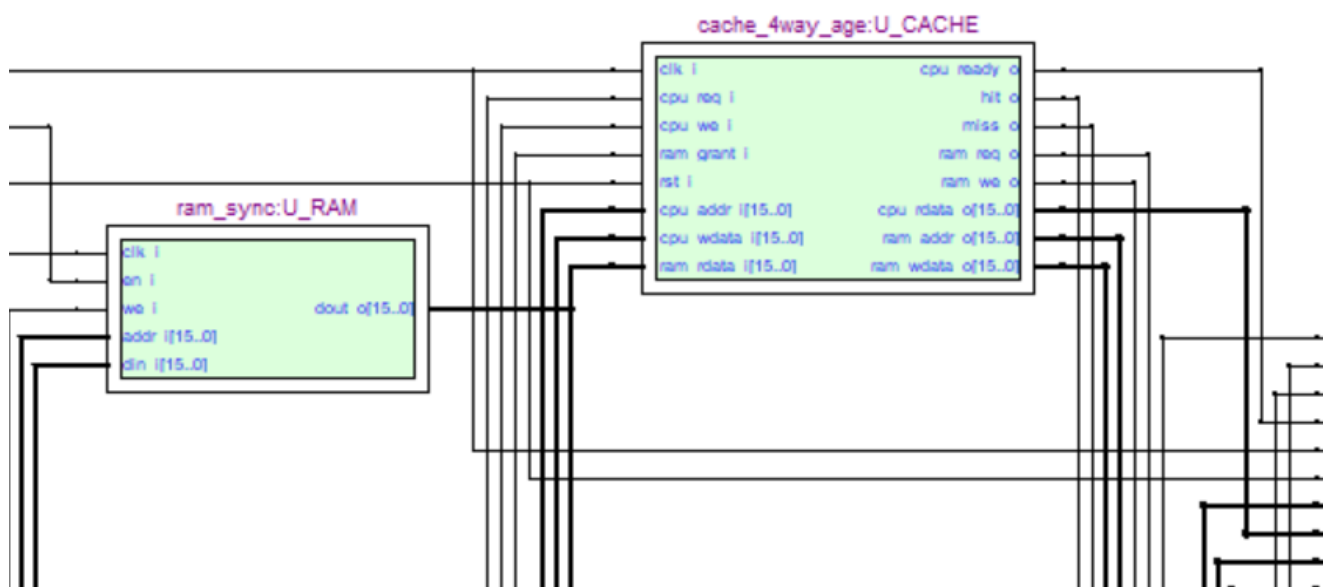


Рисунок 2.2 - Символическое представление модулей ПЗУ, ОЗУ и кэша данных

ПЗУ команд работает только в режиме чтения. На его входы поступают тактовый сигнал `clk_i`, сигнал разрешения `en_i` и адрес `addr_i[15:0]`, формируемый указателем команд IP. На активном фронте такта при `en_i = 1` адрес фиксируется, после чего на выходе `data_o[15:0]` появляется слово команды. Так как каждая команда занимает два 16-разрядных слова, устройство управления последовательно выбирает слово IR0 по адресу IP и слово IR1 по адресу IP + 1.

ОЗУ данных работает в режимах чтения и записи. На его входы поступают `clk_i`, `en_i`, `we_i`, адрес `addr_i[15:0]` и данные записи `din_i[15:0]`; на выходе формируется `dout_o[15:0]`. При `en_i = 1` и `we_i = 1` на активном фронте такта выполняется запись слова в выбранную ячейку. При `en_i = 1` и `we_i = 0` выполняется чтение, а считанное слово появляется на выходе после синхронного обращения.

Кэш данных предназначен для ускорения обращений процессора к ОЗУ. Он имеет организацию 4-way, 16 наборов, строку размером одно 16-разрядное слово и использует признаки действительности, теги и счётчики давности хранения. При попадании данные возвращаются процессорному ядру без обращения к ОЗУ. При промахе кэш запрашивает доступ к памяти через арбитр, получает слово из ОЗУ и заполняет выбранную строку. Запись выполняется по принципу write-through: новое значение передаётся и в кэш, и в ОЗУ.

Так как к ОЗУ могут обращаться кэш процессора и КПДП, доступ к памяти согласуется арбитром. Арбитр выдаёт сигналы `grant_cpu` и `grant_dma`; при одновременных запросах приоритет получает КПДП. Это предотвращает конфликт на адресной шине, шине данных и сигнале записи ОЗУ.

Таблица 2.1 - Основные сигналы запоминающих устройств

Блок	Сигнал	Назначение
ПЗУ	<code>clk_i</code>	тактовый сигнал синхронного чтения
ПЗУ	<code>en_i</code>	разрешение чтения слова программы
ПЗУ	<code>addr_i[15:0]</code>	адрес слова команды
ПЗУ	<code>data_o[15:0]</code>	слово команды для загрузки в IR0 или IR1
ОЗУ	<code>clk_i</code>	тактовый сигнал синхронного чтения/записи
ОЗУ	<code>en_i</code>	разрешение обращения к памяти данных
ОЗУ	<code>we_i</code>	выбор режима записи
ОЗУ	<code>addr_i[15:0]</code>	адрес слова данных
ОЗУ	<code>din_i[15:0]</code>	данные, записываемые в ОЗУ
ОЗУ	<code>dout_o[15:0]</code>	данные, считанные из ОЗУ
Кэш	<code>cpu_req_i</code>	запрос процессора к кэшу
Кэш	<code>cpu_we_i</code>	признак записи
Кэш	<code>cpu_ready_o</code>	готовность результата обращения
Кэш	<code>hit_o, miss_o</code>	признаки попадания и промаха
Арбитр	<code>req_cpu_i, req_dma_i</code>	запросы от процессора и КПДП
Арбитр	<code>grant_cpu_o, grant_dma_o</code>	разрешения доступа к ОЗУ

Таблица 2.2 - Начальные данные ОЗУ

Адрес	Значение	Назначение
000Ah	0000h	область записи КПДП
000Bh	0000h	область записи КПДП
000Ch	0000h	область записи КПДП
0020h	8001h	операнд для загрузки в R1

Адрес	Значение	Назначение
0021h	00F0h	операнд для загрузки в R2
0022h	0F0Fh	операнд для операции OR
0023h	0000h	операнд для операции NOR
0024h	1234h	контрольное слово для проверяемых переходов
0025h	FFFFh	операнд для формирования нулевого результата
0030h	0000h	ячейка для записи результата

Для временной диаграммы работы ОЗУ необходимо показать два основных режима: чтение и запись. При чтении кэш или КПДП формирует адрес, арбитр выдаёт разрешение, после чего $en_i = 1$, $we_i = 0$, а считанное слово появляется на $dout_o$. При записи одновременно выставляются адрес, din_i , $en_i = 1$ и $we_i = 1$; на активном фронте такта данные заносятся в выбранную ячейку. Если доступ к памяти запрашивают процессор и КПДП одновременно, на диаграмме должно быть видно, что $grant_dma_o = 1$, а $grant_cpu_o = 0$ до завершения обращения КПДП [7].

Результаты функциональной проверки подсистемы памяти приведены на рисунках 2.3-2.5. На диаграмме отдельно показаны запрос контроллера прямого доступа к памяти, выдача арбитром разрешения, запись трёх контрольных слов в ОЗУ и установка признака завершения передачи.

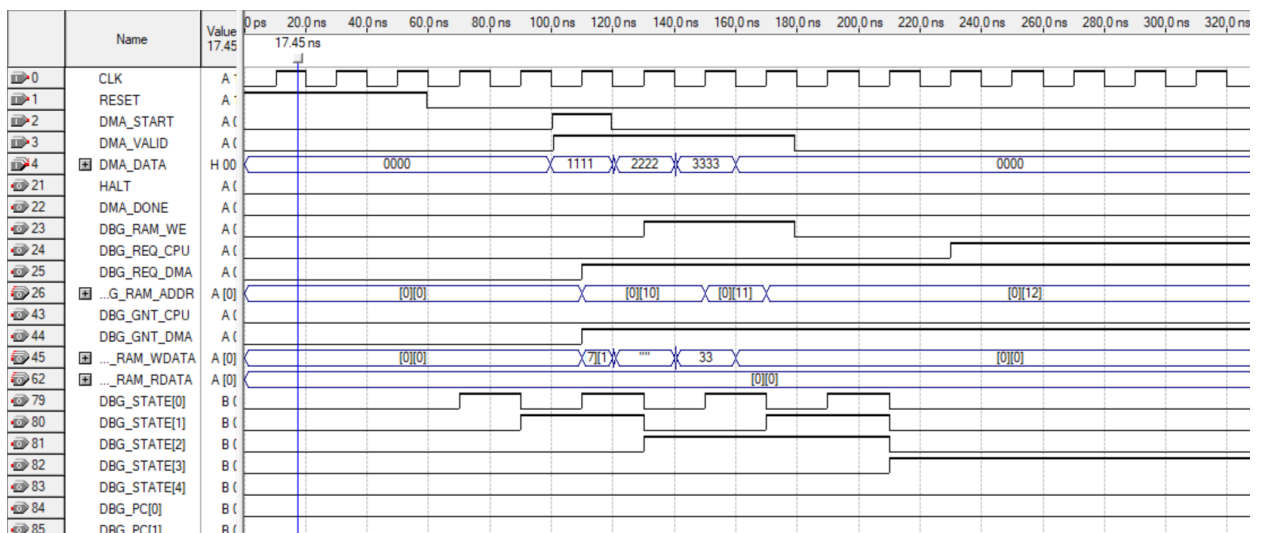


Рисунок 2.3 - Временная диаграмма работы КПДП и ОЗУ

На рисунке 2.3 видно, что после активации START контроллер формирует запрос REQ. После получения GNT по адресам 000Ah, 000Bh и

000Ch последовательно записываются слова 1111h, 2222h и 3333h. По окончании передачи устанавливается сигнал DONE.

Addr	+0	+1	+2	+3	+4	+5	+6	+7
00	0000	0000	0000	0000	0000	0000	0000	0000
08	0000	0000	0000	0000	0000	0000	0000	0000
10	0000	0000	0000	0000	0000	0000	0000	0000
18	0000	0000	0000	0000	0000	0000	0000	8001
20	00F0	0F0F	0000	1234	FFFF	0000	0000	0000
28	0000	0000	0000	0000	0000	0000	0000	0000
30	0000	0000	0000	0000	0000	0000	0000	0000

Рисунок 2.4 - Контрольный дамп ОЗУ после выполнения операций обмена

Addr	+0	+1	+2	+3	+4	+5	+6	+7
00	0000	0000	0000	0000	0000	0000	0000	0000
08	0000	0000	1111	2222	3333	0000	0000	0000
10	0000	0000	0000	0000	0000	0000	0000	0000
18	0000	0000	0000	0000	0000	0000	0000	0000
20	8001	00F0	0F0F	0000	1234	FFFF	0000	0000
28	0000	0000	0000	0000	0000	0000	0000	0000
30	C001	0000	0000	0000	0000	0000	0000	0000

Рисунок 2.5 - Контрольный дамп ОЗУ после выполнения операций обмена

Сравнение дампов подтверждает, что область обмена КППД была изменена только по адресам 000Ah-000Ch, а процессорная часть записала результат C001h в ячейку 0030h. Остальные контрольные ячейки, используемые как операнды команд, сохраняют исходные значения.

Содержимое файла rom_program.mif задаёт начальную программу, размещаемую в ПЗУ команд. В данной реализации слово ПЗУ имеет разрядность 16 бит, поэтому каждое слово команды хранится в отдельной ячейке памяти. Первое слово команды содержит код операции и номер регистра, второе слово содержит адресную часть либо резервное значение.

Содержимое файла rom_program.mif

1. WIDTH=16;
2. DEPTH=256;
3. ADDRESS_RADIX=HEX;
4. DATA_RADIX=HEX;
- 5.

```

6. CONTENT BEGIN
7.  [00..FF] : 0000;
8.
9.  -- Загрузка операнда в R1
10. 00 : 0110; -- MOV 0020h, R1
11. 01 : 0020;
12.
13. -- Арифметический сдвиг и инкремент по флагу S
14. 02 : 0510; -- SRA R1
15. 03 : 0000;
16. 04 : 0610; -- INCS R1
17. 05 : 0000;
18.
19. -- Работа со стеком
20. 06 : 0710; -- PUSH R1
21. 07 : 0000;
22.
23. -- Загрузка операнда в R2 и логические операции
24. 08 : 0120; -- MOV 0021h, R2
25. 09 : 0021;
26. 0A : 0320; -- OR R2, 0022h
27. 0B : 0022;
28. 0C : 0420; -- NOR R2, 0023h
29. 0D : 0023;
30.
31. -- Извлечение из стека и запись результата в память
32. 0E : 0830; -- POP R3
33. 0F : 0000;
34. 10 : 0230; -- MOV R3, 0030h
35. 11 : 0030;
36.
37. -- Формирование нулевого результата для проверки JZ
38. 12 : 0140; -- MOV 0025h, R4
39. 13 : 0025;
40. 14 : 0440; -- NOR R4, 0025h
41. 15 : 0025;
42.
43. -- Условный переход
44. 16 : 0A00; -- JZ 001Ah
45. 17 : 001A;
46.
47. -- Команда должна быть пропущена при успешном переходе
48. 18 : 0150; -- MOV 0024h, R5
49. 19 : 0024;
50.
51. -- Безусловный переход
52. 1A : 0900; -- JMP 001Eh
53. 1B : 001E;
54.
55. -- Команда должна быть пропущена после JMP
56. 1C : 0160; -- MOV 0024h, R6
57. 1D : 0024;
58.
59. -- Чтение области, изменяемой КППД
60. 1E : 0170; -- MOV 000Ah, R7

```

```

61. 1F : 000A;
62.
63. -- Останов программы
64. 20 : 0000; -- HLT
65. 21 : 0000;
66. END;

```

2.2 Устройство управления

Устройство управления является центральным узлом микро-ЭВМ. Оно выполняет выборку команды из ПЗУ, загрузку слов команды в регистры IR0 и IR1, декодирование кода операции, выбор исполнительного маршрута и формирование управляющих сигналов для регистрового файла, АЛУ, стека, кэша, ПЗУ, предсказателя переходов и указателя команд IP.

В проекте устройство управления реализовано как синхронный конечный автомат с жёсткой логикой. Его состояние хранится в регистре `state_r`. Переходы между состояниями выполняются по активному фронту `clk_i`, а при `rst_i = 1` автомат переходит в начальное состояние, очищает регистры команды и устанавливает указатель команд в нулевой адрес.

В состав устройства управления входят:

- регистр указателя команд `pc_r`;
- регистры команды `ir0_r` и `ir1_r`;
- регистр текущего кода операции `cur_opcode_r`;
- регистр номера выбранного РОН `cur_reg_r`;
- логика декодирования команд;
- логика выбора следующего состояния автомата;
- формирователь сигналов обращения к ПЗУ, кэшу, РОН, АЛУ, стеку и предсказателю переходов.

Схемы декодера команд, счётчика фаз и блока IP/IR приведены в приложениях Б, В и Г.

На схеме верхнего уровня устройство управления входит в состав процессорного ядра `cpu_core`, которое связано с ПЗУ команд, кэшем данных, блоком РОН, АЛУ, стеком, регистром флагов и предсказателем переходов.

Назначение основных элементов схемы блока УУ:

- `state_r` – регистр текущего состояния конечного автомата устройства управления;
- `pc_r` – указатель команд, содержащий адрес первого слова текущей команды;

- ir0_r – регистр первого слова команды, содержащего код операции и номер регистра;
- ir1_r – регистр второго слова команды, содержащего адрес операнда или адрес перехода;
- cur_opcode_r – регистр текущего кода операции, используемый при выборе маршрута выполнения команды;
- cur_reg_r – регистр номера выбранного регистра общего назначения;
- flags_reg – регистр флагов Z, S, C, O, обновляемый после операций АЛУ;
- stack7x16 – стековый блок, обслуживающий команды PUSH и POP;
- branch_predictor – блок сохранения истории и результата условного перехода;
- логика переходов автомата – формирование последовательности состояний выборки, декодирования и выполнения команды.

Назначение входных пинов блока:

- CLK – тактовый сигнал;
- RESET – сигнал сброса устройства управления и исполнительных блоков;
- DMA_START – запуск передачи данных контроллером прямого доступа к памяти;
- DMA_VALID – признак достоверности входного слова DMA;
- DMA_DATA[15..0] – входная шина данных внешнего устройства.
- Назначение выходных и отладочных пинов блока:
- HALT – признак останова процессора;
- DMA_DONE – признак завершения передачи трёх слов КПДП;
- DBG_STATE[4..0] – код текущего состояния автомата УУ;
- DBG_PC[15..0] – текущее значение указателя команд;
- DBG_IR0[15..0] – первое слово текущей команды;
- DBG_IR1[15..0] – второе слово текущей команды;
- DBG_R1..DBG_R7 – контрольные значения регистров общего назначения;
- DBG_FLAGS[3..0] – флаги Z, S, C, O;
- DBG_SP[2..0] – указатель вершины стека;
- DBG_RAM_WE – признак записи в ОЗУ;
- DBG_RAM_ADDR[15..0] – адрес обращения к ОЗУ;
- DBG_RAM_WDATA[15..0] – данные, записываемые в ОЗУ;

– DBG_BP_HIST[1..0] – регистр истории предсказателя условных переходов.

2.2.1 Декодирование команд

После загрузки двух слов команды автомат переходит в состояние декодирования. Код операции берётся из старшего байта IR0, то есть из поля IR0[15:8]. Номер регистра берётся из поля IR0[7:4]. Второе слово IR1 используется как адрес операнда в ОЗУ или как адрес перехода.

Таблица 2.3 - Декодирование команд устройством управления

Команда	КОП	Основной маршрут выполнения
HLT	00h	переход в состояние останова
MOV adr, Rn	01h	чтение из кэша/ОЗУ, запись в РОН
MOV Rn, adr	02h	чтение РОН, запись через кэш в ОЗУ
OR Rn, adr	03h	чтение памяти, операция АЛУ, запись результата
NOR Rn, adr	04h	чтение памяти, операция АЛУ, запись результата
SRA Rn	05h	операция АЛУ над регистром
INCS Rn	06h	операция АЛУ с использованием флага S
PUSH Rn	07h	запись регистра в стек
POP Rn	08h	чтение из стека, запись в РОН
JMP adr	09h	загрузка IP адресом из IR1
JZ adr	0Ah	проверка флага Z и условная загрузка IP

Любой код операции, не входящий в указанный набор, переводит устройство управления в состояние останова. Такое решение упрощает отладку и исключает случайное выполнение неопределённой команды.

2.2.2 Фазы выполнения команды

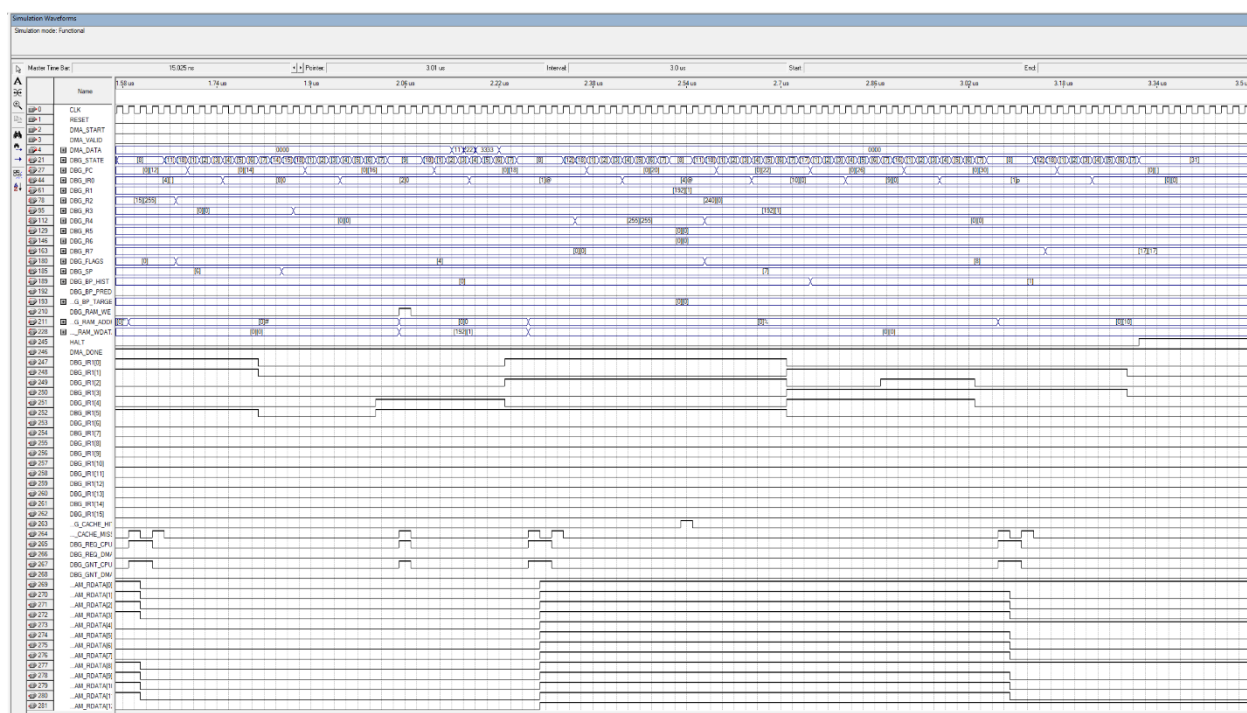
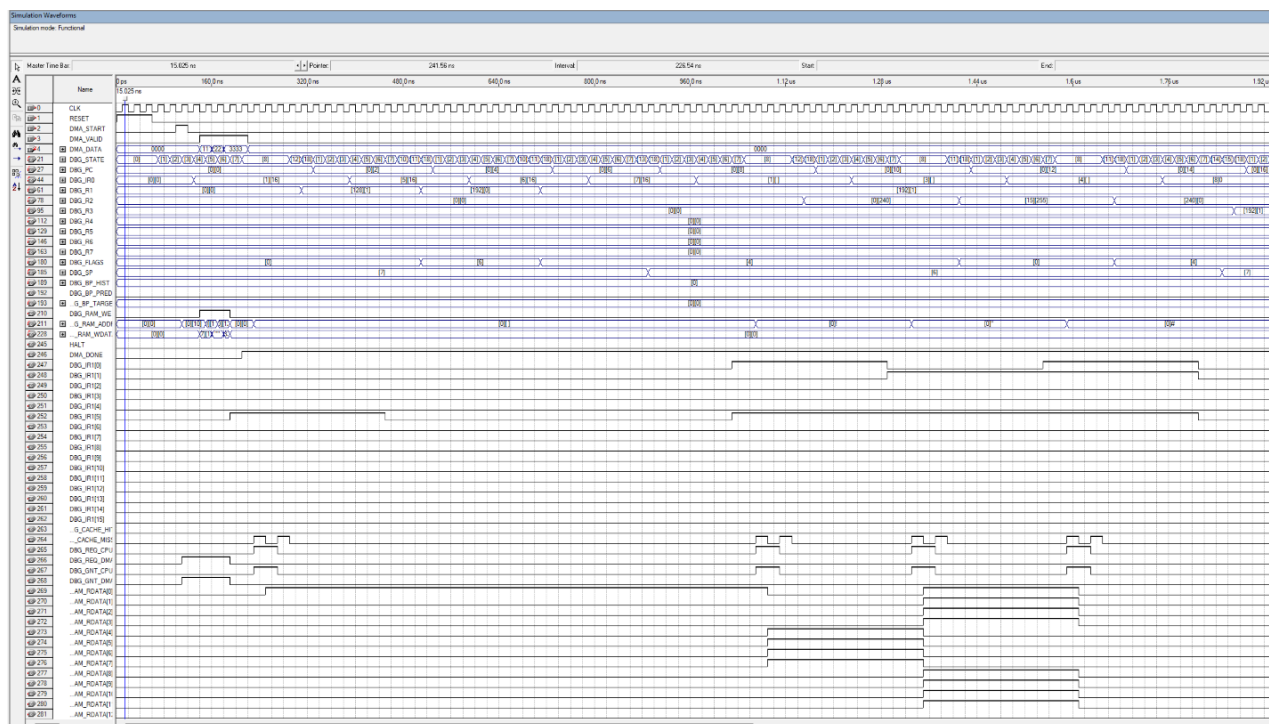
Работу автомата удобно представить как последовательность фаз. В проектной модели эти фазы реализованы отдельными состояниями автомата.

Таблица 2.4 - Основные фазы выполнения команды

Фаза	Основные состояния автомата	Назначение
T0	S_FETCH0_REQ	выдача адреса IP в ПЗУ
T1	S_FETCH0_WAIT; S_FETCH0_LATCH	ожидание и загрузка IR0
T2	S_FETCH1_REQ; S_FETCH1_WAIT; S_FETCH1_LATCH	выборка и загрузка IR1
T3	S_DECODE	декодирование КОП и номера регистра
T4	S_CACHE_READ; S_CACHE_WRITE; S_ALU_SETUP_REG; S_STACK_PUSH; S_STACK_POP; S_BRANCH_JMP; S_BRANCH_JZ	выполнение операции
Tw	ожидание cache_ready_i или grant	ожидание памяти, кэша или арбитра
T5	S_ALU_WRITE; S_RF_WRITE_MEM; S_RF_WRITE_POP; S_FINISH	запись результата, обновление флагов и переход к следующей команде

При обычном линейном выполнении после завершения команды указатель команд увеличивается на два, так как каждая команда занимает два 16-разрядных слова. Для команды JMP в IP загружается адрес из IR1. Для команды JZ сначала проверяется флаг Z: если $Z = 1$, выполняется переход по адресу из IR1, иначе загружается адрес следующей команды.

Временная диаграмма работы полной модели показывает последовательную выборку двухсловных команд, изменение указателя команд, загрузку регистров IR0 и IR1, изменение регистров общего назначения, обращения к памяти и завершение программы командой HLT.



На рисунке 2.6 показано, что IP последовательно принимает адреса первых слов команд, IR0 содержит код операции и номер регистра, а IR1 - адресную часть. После выполнения арифметико-логических и стековых команд регистры R1, R2, R3, R4 и R7 принимают контрольные значения, флаг

Z устанавливается после нулевого результата, а сигнал HALT фиксирует окончание программы.

Номер пункта соответствует наблюдаемой контрольной точке:

1. После передачи КПДП сигнал DMA_DONE устанавливается в 1, что подтверждает завершение записи трёх слов в ОЗУ.
2. Регистр R1 последовательно принимает значения 0000h -> 8001h -> C000h -> C001h. Это подтверждает выполнение команд MOV, SRA и INCS.
3. Регистр R2 последовательно принимает значения 0000h -> 00F0h -> 0FFFh -> F000h. Это подтверждает выполнение логических операций OR и NOR.
4. Указатель стека DBG_SP изменяется 7 -> 6 -> 7, что подтверждает корректное выполнение команд PUSH и POP.
5. После выполнения POP в регистр R3 загружается значение C001h. Затем это значение записывается в ОЗУ по адресу 0030h.
6. Регистр R4 изменяется FFFFh -> 0000h. После получения нулевого результата в DBG_FLAGS устанавливается значение 1000, что соответствует активному флагу Z.
7. При выполнении команды JZ указатель команд переходит с адреса 0016h на адрес 001Ah. Следовательно, условный переход выполнен успешно.
8. Регистры R5 и R6 сохраняют значение 0000h, так как команды по пропущенным адресам не выполняются.
9. Регистр R7 получает значение 1111h из области обмена DMA.
10. DBG_BP_HIST принимает значение 01, что подтверждает обновление истории условного перехода.
11. В конце моделирования сигнал HALT устанавливается в 1, что подтверждает корректное завершение программы.

2.2.3 Основные управляющие сигналы

Устройство управления формирует управляющие сигналы для всех исполнительных блоков. Часть сигналов является внешней по отношению к процессорному ядру, часть используется внутри cpi_core.

Таблица 2.5 - Основные управляющие сигналы устройства управления

Сигнал	Приёмник	Назначение
rom_en_o	ПЗУ	разрешение выборки слова команды
rom_addr_o[15:0]	ПЗУ	адрес слова команды

Сигнал	Приёмник	Назначение
cache_req_o	кэш	запрос чтения или записи данных
cache_we_o	кэш	признак записи в память данных
cache_addr_o[15:0]	кэш	адрес операнда в ОЗУ
cache_wdata_o[15:0]	кэш	данные для записи
rf_we	РОН	разрешение записи результата
rf_wr_addr	РОН	номер регистра назначения
alu_op_r[2:0]	АЛУ	код операции АЛУ
flag_we	регистр FR	разрешение обновления флагов
stack_push	стек	запись слова в стек
stack_pop	стек	чтение слова из стека
bp_query_o	предсказатель	запрос прогноза перехода
bp_update_o	предсказатель	обновление результата перехода
halt_o	верхний уровень	признак останова процессора
dbg_state_o[4:0]	отладочная шина	код текущего состояния автомата
dbg_pc_o[15:0]	отладочная шина	значение указателя команд
dbg_ir0_o[15:0]	отладочная шина	первое слово текущей команды
dbg_ir1_o[15:0]	отладочная шина	второе слово текущей команды
dbg_flags_o[3:0]	отладочная шина	состояние флагов Z, S, C, O
dbg_sp_o[2:0]	отладочная шина	указатель вершины стека
dbg_bp_hist_o[1:0]	отладочная шина	история условных переходов

2.2.4 Указатель команд и регистры команды

Указатель команд `pc_r` хранит адрес первого слова текущей команды. После сброса он устанавливается в `0000h`. При линейном выполнении команд в конце цикла вычисляется $PC + 2$. Такое увеличение связано с фиксированной длиной команды: первое слово содержит код операции и номер регистра, второе слово содержит адресную часть.

Регистр `IR0` хранит первое слово команды. Из него выделяются код операции `IR0[15:8]` и номер регистра `IR0[7:4]`. Регистр `IR1` хранит второе слово команды и используется как адрес операнда в ОЗУ или как адрес перехода в ПЗУ команд.

2.2.5 Управление АЛУ, стеком и памятью

Для команд `OR` и `NOR` устройство управления сначала запускает чтение второго операнда из памяти через кэш. После появления `cache_ready_i = 1` данные передаются на вход АЛУ, выполняется операция, результат записывается в РОН, а флаги `Z`, `S`, `C` и `O` фиксируются в регистре `FR`.

Для команд `SRA` и `INCS` обращение к памяти не требуется. Устройство управления выбирает регистр `Rn`, передаёт его значение на вход АЛУ, задаёт код операции и затем записывает результат обратно в тот же регистр.

Для команды `PUSH` активируется сигнал `stack_push`, и содержимое выбранного регистра заносится в стек. Для команды `POP` активируется `stack_pop`, затем считанное слово записывается в выбранный регистр общего назначения. Состояния пустого и полного стека учитываются аппаратной логикой блока стека.

При выполнении `MOV adr, Rn` устройство управления запускает чтение из кэша/ОЗУ, а после готовности данных записывает слово в выбранный РОН. При выполнении `MOV Rn, adr` значение регистра передаётся в кэш, а далее через механизм `write-through` попадает в ОЗУ данных.

2.2.6 Останов и взаимодействие с предсказателем переходов

Команда `HLT` переводит автомат в состояние `S_HALT`. В этом состоянии процессор больше не выполняет выборку новых команд, а на выходе `halt_o` устанавливается активный уровень.

Предсказатель переходов запрашивается при выборке команды через сигнал `bp_query_o`. При выполнении условного перехода `JZ` устройство

управления передаёт фактический результат перехода через `bp_update_o`, `bp_taken_o` и `bp_target_o`. Если переход выполняется, в IP загружается адрес из `IR1`; если нет, выполнение продолжается с адреса `IP + 2`.

Таким образом, устройство управления задаёт полный цикл работы микро-ЭВМ: выбирает команду из ПЗУ, декодирует её, активирует нужный исполнительный блок, ожидает готовность кэша или арбитра при необходимости, записывает результат и формирует адрес следующей команды. При выполнении команды `HLT` автомат переходит в состояние `S_HALT`. В этом состоянии новые команды из ПЗУ не выбираются, управляющие воздействия на исполнительные блоки не формируются, а выходной сигнал `halt_o` удерживается в активном состоянии. Это состояние является конечным при выполнении команды `HLT` и подтверждает завершение функционального моделирования.

2.3 Регистры общего назначения

Блок регистров общего назначения предназначен для хранения промежуточных данных и результатов выполнения команд. В разработанной микро-ЭВМ блок `РОН` содержит 12 регистров `R0-R11` разрядностью 16 бит. В данной реализации значения регистров `R1-R7` выведены на отладочные шины `DBG_R1-DBG_R7` для контроля работы системы. Структурная схема блока регистров общего назначения приведена в приложении Е.

Поле номера регистра в команде имеет разрядность 4 бита, поэтому теоретически позволяет адресовать 16 регистров. В данной реализации используются только коды 0-11, соответствующие регистрам `R0-R11`; коды 12-15 считаются резервными. Если при записи номер регистра выходит за допустимый диапазон, запись не выполняется.

Запись в регистр выполняется по активному фронту тактового сигнала при `we_i = 1`. Номер записываемого регистра задается шиной `wr_addr_i[3..0]`, а записываемое слово поступает по шине `din_i[15..0]`. При `rst_i = 1` содержимое всех регистров сбрасывается в `0000h`.

Чтение данных из блока `РОН` выполняется по двум независимым адресным входам `rd_addr_a_i` и `rd_addr_b_i`.

На выходы `dout_a_o` и `dout_b_o` поступают значения выбранных регистров. Первый канал используется для передачи операнда в АЛУ или данных на запись в память, а второй канал оставлен для операций, которым может потребоваться второй регистровый операнд.

Назначение основных элементов блока РОН:

- массив regs - хранение двенадцати 16-разрядных регистров общего назначения;
- wr_addr_i[3..0] - выбор регистра для записи;
- rd_addr_a_i[3..0], rd_addr_b_i[3..0] - выбор регистров для чтения;
- we_i - разрешение записи;
- din_i[15..0] - входная шина данных;
- dout_a_o[15..0], dout_b_o[15..0] - выходные шины чтения;
- r0_o-r11_o - отладочные выходы содержимого регистров.

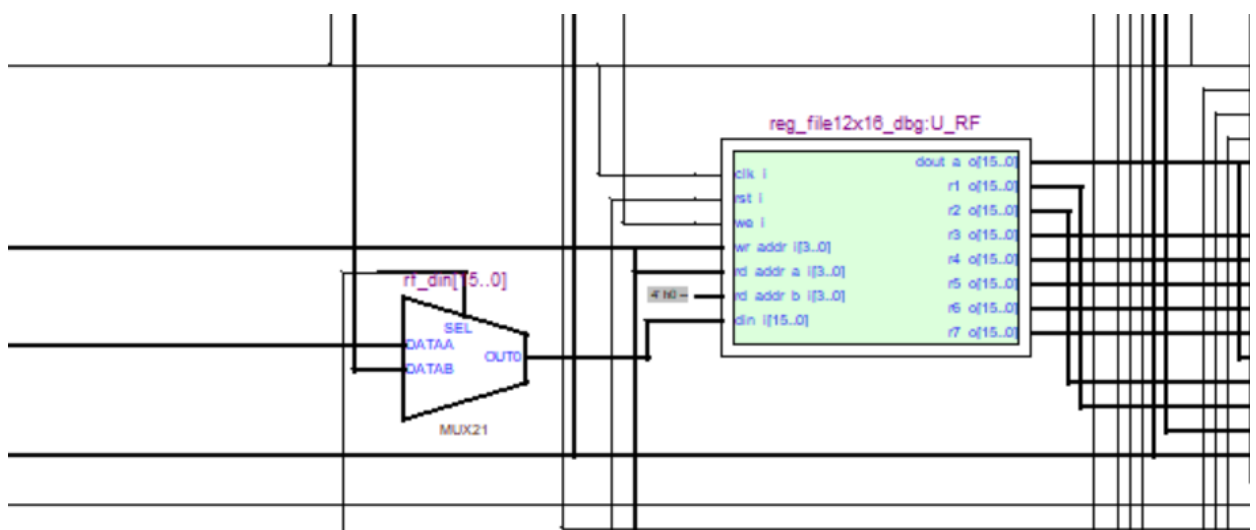


Рисунок 2.8 - Расположение блока регистров общего назначения в структуре процессорного ядра

2.4 Блок общих операций

Блок выполнения общих операций процессорного ядра предназначен для организации полного цикла работы микро-ЭВМ при выполнении команды. Он обеспечивает выборку команды из ПЗУ, загрузку регистров команды, декодирование кода операции, выбор исполнительного маршрута, формирование обращений к памяти данных, запуск арифметико-логического устройства, работу со стеком, изменение указателя команд и останов процессора.

Внутри сru_core находятся указатель команд pc_r, регистры команды ir0_r и ir1_r, регистровый файл общего назначения, АЛУ, регистр флагов, стек, а также конечный автомат управления state_r.

Расположение блока выполнения операций в структуре процессорного ядра показано на рисунке 2.9.

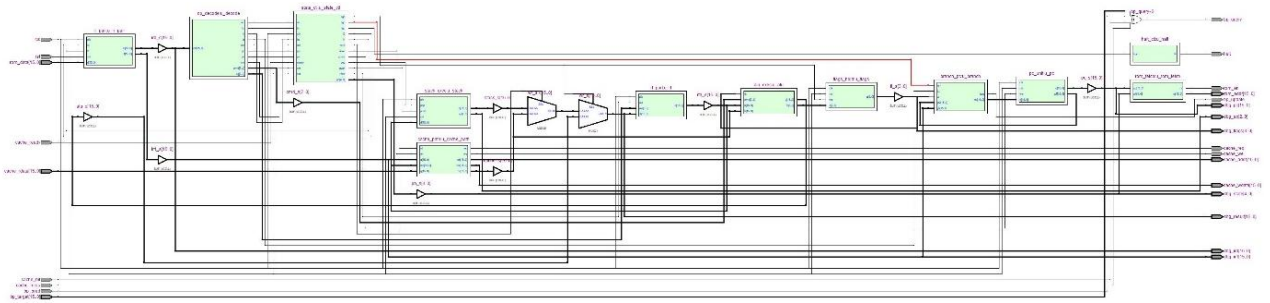


Рисунок 2.9 - Расположение блока выполнения операций в структуре процессорного ядра

Состав блока и назначение основных элементов схемы:

- state_r - конечный автомат управления, задающий текущий этап выборки, декодирования и выполнения команды;
- pc_r - указатель команды, содержащий адрес первого слова текущей или следующей команды в ПЗУ;
- ir0_r, ir1_r - регистры команды, в которые загружаются первое и второе 16-разрядные слова команды;
- cur_opcode_r - регистр декодированного кода операции;
- cur_reg_r - регистр номера РОН, выбранного полем REG_NUM;
- U_RF - блок регистров общего назначения R0-R11;
- U_ALU - арифметико-логическое устройство, выполняющее операции OR, NOR, SRA, INCS;
- регистр флагов Z, S, C, O - хранение признаков результата операций;
- U_STACK - стек данных глубиной 7 слов с направлением роста вниз;
- интерфейс ПЗУ команд - выборка двух слов команды;
- интерфейс кэш-памяти данных - выполнение операций чтения и записи данных;
- интерфейс предсказателя переходов - запрос прогноза и обновление таблицы переходов.

Далее описано назначение внешних входных пинов блока:

- clk_i - тактовый сигнал процессорного ядра;
- rst_i - сигнал синхронного сброса автомата и регистров;
- rom_data_i[15..0] - слово команды, считываемое из ПЗУ команд;
- cache_rdata_i[15..0] - слово данных, полученное из кэш-памяти;
- cache_ready_i - признак завершения операции чтения или записи через кэш;

- cache_hit_i - признак попадания при обращении к кэш-памяти;
- cache_miss_i - признак промаха при обращении к кэш-памяти;
- bp_hist_i[1..0] - история переходов, поступающая от предсказателя;
- bp_pred_i - предсказанное направление условного перехода;
- bp_target_i[15..0] - предсказанный адрес перехода.

Далее описано назначение внешних выходных пинов блока:

- rom_addr_o[15..0] - адрес слова команды в ПЗУ;
- rom_en_o - разрешение обращения к ПЗУ команд;
- cache_req_o - запрос операции чтения или записи данных через кэш;
- cache_we_o - выбор режима обращения к кэш-памяти: чтение или запись;
- cache_addr_o[15..0] - адрес операнда в памяти данных;
- cache_wdata_o[15..0] - данные, передаваемые на запись в память;
- bp_query_o - запрос к предсказателю переходов;
- bp_pc_query_o[15..0] - адрес команды, для которой запрашивается прогноз;
- bp_update_o - сигнал обновления предсказателя после фактического выполнения перехода;
- bp_pc_update_o[15..0] - адрес команды перехода, по которой обновляется предсказатель;
- bp_hist_o[1..0] - история переходов, передаваемая при обновлении;
- bp_taken_o - фактический результат условного перехода;
- bp_target_o[15..0] - фактический адрес перехода;
- halt_o - признак останова выполнения программы;
- dbg_state_o[4..0], dbg_pc_o[15..0], dbg_ir0_o[15..0], dbg_ir1_o[15..0] - отладочные выходы текущего состояния, указателя команд и регистров команды;
- dbg_r1_o-dbg_r7_o - отладочные выходы регистров общего назначения;
- dbg_flags_o[3..0] - отладочный выход регистра флагов;
- dbg_sp_o[2..0] - отладочный выход указателя стека.

Внутренние управляющие сигналы исполнительной части:

- rf_we - разрешение записи в блок РОН;
- rf_wr_addr[3..0] - номер регистра, в который записывается результат;
- rf_rd_a[3..0], rf_rd_b[3..0] - номера регистров, считываемых из РОН;
- rf_din[15..0] - данные, записываемые в выбранный регистр;
- alu_a_r[15..0], alu_b_r[15..0] - операнды АЛУ;

- alu_op_r[2..0] - код операции АЛУ;
- alu_y[15..0] - результат АЛУ;
- flag_we - разрешение обновления регистра флагов;
- stack_push, stack_pop - сигналы занесения слова в стек и извлечения слова из стека;
- stack_dout[15..0] - слово, считанное из стека;
- stack_sp[2..0] - текущий указатель стека.

Входные и выходные связи блока выполнения операций можно разделить на несколько групп:

- интерфейс ПЗУ команд: rom_en_o, rom_addr_o, rom_data_i;
- интерфейс кэш-памяти
данных: cache_req_o, cache_we_o, cache_addr_o, cache_wdata_o, cache_rdata_i, cache_ready_i;
- интерфейс предсказателя
переходов: bp_query_o, bp_pc_query_o, bp_update_o, bp_taken_o, bp_target_o;
- отладочные
выходы: dbg_state_o, dbg_pc_o, dbg_ir0_o, dbg_ir1_o, dbg_r1_o-
dbg_r7_o, dbg_flags_o, dbg_sp_o;
- сигнал останова программы halt_o.

Общий алгоритм работы блока задаётся конечным автоматом state_r. После сброса автомат переходит к выборке команды. Так как команда имеет фиксированную длину два 16-разрядных слова, выборка всегда выполняется одинаково: первое слово помещается в IR0, второе слово помещается в IR1. После этого выполняется декодирование кода операции OPCODE[7..0], расположенного в старших разрядах IR0.

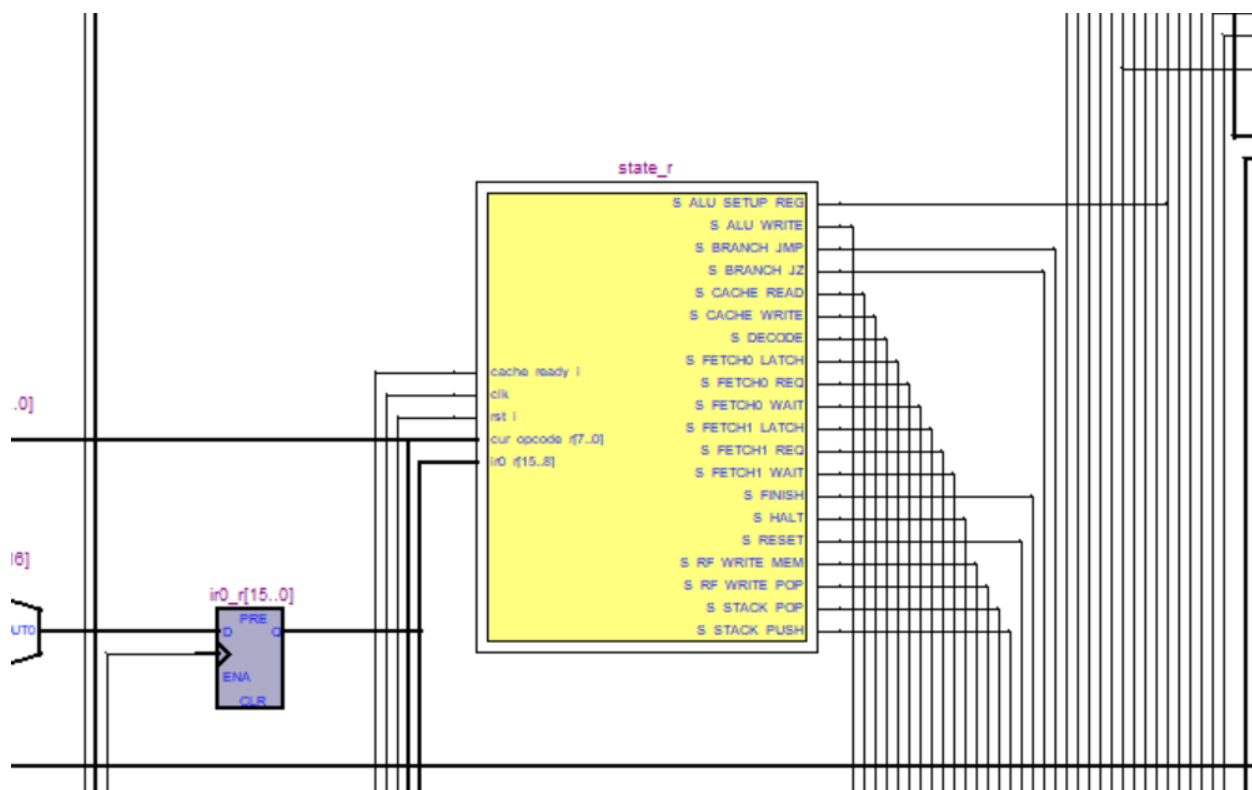


Рисунок 2.10 - Автомат выполнения общих операций процессорного ядра.

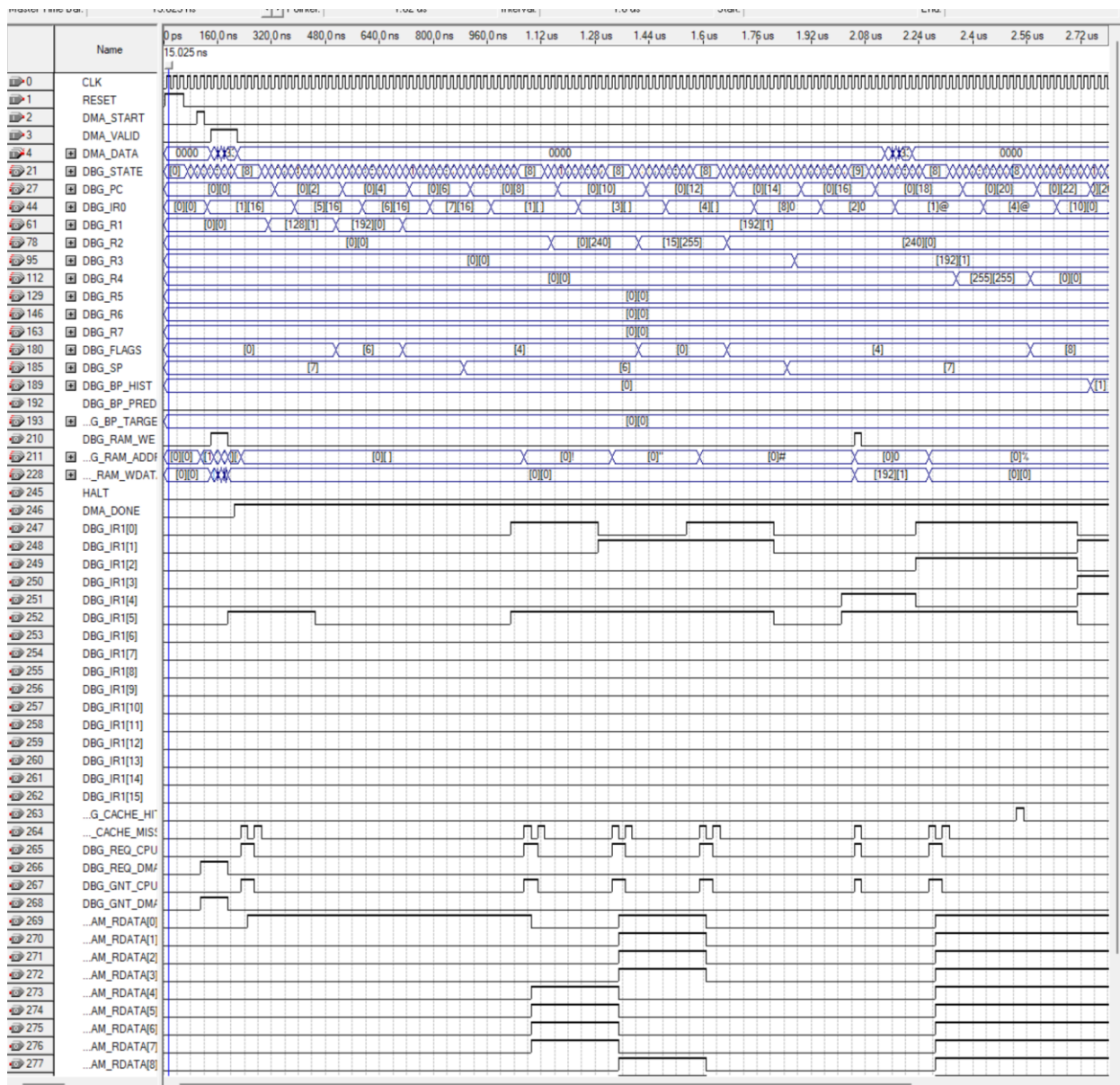


Рисунок 2.11 - Временная диаграмма выполнения команд процессорным ядром микро-ЭВМ

Таблица 2.6 - Основные состояния автомата выполнения команд

Состояние	Назначение
S_RESET	начальная установка регистров и переход к выборке команды
S_FETCH0_REQ	выдача адреса первого слова команды в ПЗУ
S_FETCH0_WAIT	ожидание данных ПЗУ
S_FETCH0_LATCH	загрузка первого слова команды в IR0
S_FETCH1_REQ	выдача адреса второго слова команды в ПЗУ

Состояние	Назначение
S_FETCH1_WAIT	ожидание второго слова команды
S_FETCH1_LATCH	загрузка второго слова команды в IR1
S_DECODE	декодирование кода операции и номера регистра
S_CACHE_READ	чтение операнда из памяти данных через кэш
S_CACHE_WRITE	запись слова в память данных через кэш
S_ALU_SETUP_REG	подготовка операнда для операций SRA и INCS
S_ALU_WRITE	запись результата АЛУ в РОН и обновление флагов
S_RF_WRITE_MEM	запись слова, считанного из памяти, в РОН
S_STACK_PUSH	занесение регистра в стек
S_STACK_POP	извлечение слова из стека
S_RF_WRITE_POP	запись извлечённого слова в РОН
S_BRANCH_JMP	выполнение безусловного перехода
S_BRANCH_JZ	выполнение условного перехода по флагу Z
S_FINISH	завершение команды и переход к следующей выборке
S_HALT	останов выполнения программы

2.4.1 Блок нулевой итерации

Общий цикл выполнения команды начинается с выборки первого слова из ПЗУ команд. В состоянии S_FETCH0_REQ на адресный вход ПЗУ передаётся значение PC, после чего в состояниях S_FETCH0_WAIT и S_FETCH0_LATCH слово команды загружается в регистр IR0. Затем выполняется выборка второго слова команды: адрес PC + 1 передаётся в ПЗУ, а считанное слово загружается в регистр IR1.

Первое слово команды содержит код операции и номер регистра. Второе слово используется как адрес операнда в памяти данных, адрес перехода или

резервное поле. Такой формат делает выборку регулярной: устройство управления всегда выбирает два слова, даже если конкретная команда использует только одно из них.

На структурной схеме выборки и декодирования показано, что первое слово команды загружается в регистр IR0, второе слово - в регистр IR1, после чего из IR0 выделяются код операции и номер регистра, а из IR1 - адрес операнда или перехода.

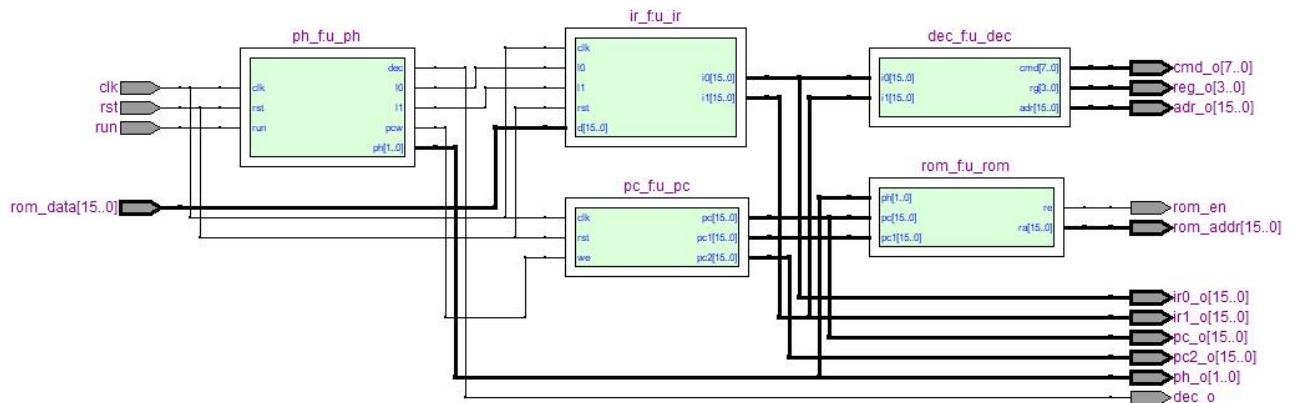


Рисунок 2.12 - Выборка и декодирование двухсловной команды

2.4.2 Блоки арбитража к общим шинам

Подключение исполнительных блоков к общим шинам выполняется через сигналы разрешения чтения и записи, формируемые устройством управления. В каждый момент времени только выбранный источник данных получает право передать значение на внутреннюю шину, а остальные блоки находятся в неактивном состоянии. Такой способ согласования исключает конфликт нескольких источников на общей магистрали и обеспечивает корректную передачу операндов между РОН, АЛУ, стеком, кэшем и ОЗУ данных.

Для операций чтения или записи данных процессор формирует сигналы к кэш-памяти:

- **cache_req_o** - запрос обращения к памяти данных;
- **cache_we_o** - направление обмена: чтение или запись;
- **cache_addr_o** - адрес операнда;
- **cache_wdata_o** - данные, записываемые в память;
- **cache_rdata_i** - данные, считанные из памяти;
- **cache_ready_i** - признак завершения обмена.

При промахе кэша обращение передаётся в ОЗУ данных через общий арбитр. Поэтому прямого конфликта между процессором и КППД на шине

ОЗУ не возникает: запросы REQ_CPU и REQ_DMA обрабатываются арбитром, а разрешения выдаются сигналами GNT_CPU и GNT_DMA.

2.4.3 Блок операции HLT (останов микро-ЭВМ)

В данной микро-ЭВМ для завершения выполнения программы используется команда HLT. Она описывается вместе с обработкой зарезервированных кодов операций, поскольку при попадании неопределённого кода автомат управления также переводит процессор в состояние останова.

Команда HLT имеет код операции 00h и предназначена для останова выполнения программы. При декодировании этой команды автомат управления переходит в состояние S_HALT. В данном состоянии выборка новых команд прекращается, а на выходе halt_o устанавливается активный уровень.

Неиспользуемые коды операций считаются зарезервированными. Если в регистр IR0 попадает код операции, не входящий в реализованный набор команд, автомат также переводится в состояние S_HALT. Это защищает модель от случайного выполнения неопределённой команды и упрощает отладку.

На структурной схеме выполнения HLT показано, что после распознавания кода 00h формируется сигнал останова, блокируется дальнейшее изменение РС и прекращается выборка новых команд.

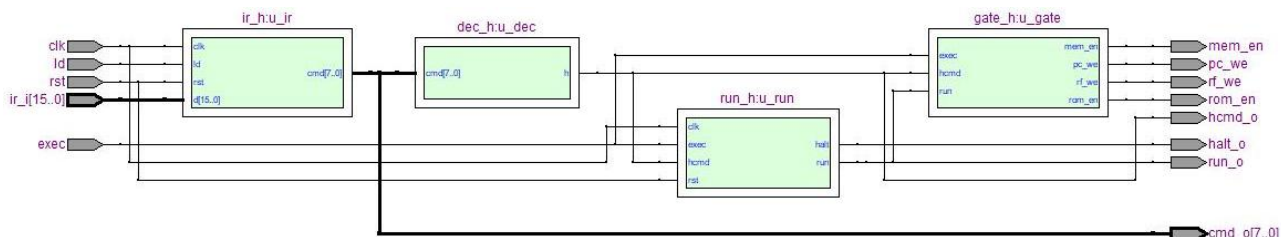


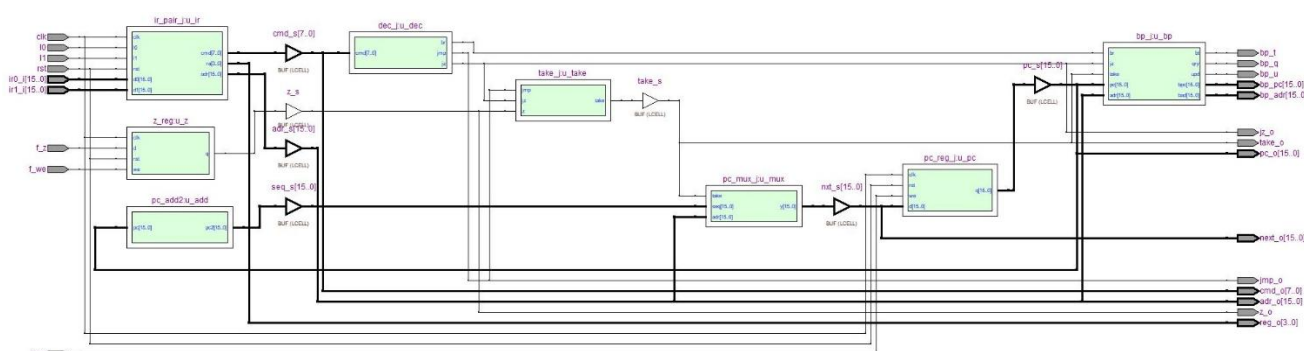
Рисунок 2.13 - Формирование сигнала останова при выполнении команды HLT

2.4.4 Блок операции JMP (безусловный переход)

Команда JMP adr имеет код операции 09h и выполняет безусловную передачу управления по адресу, заданному вторым словом команды. При выполнении этой команды значение РС не увеличивается на два, а заменяется значением IR1.

Команда JMP 001Eh кодируется словами IR0 = 0900h и IR1 = 001Eh. После её выполнения следующая выбираемая команда находится по адресу 001Eh, а команда по адресу 001Ch пропускается.

Перед командой JZ может выполняться команда NOR R4, 0025h, формирующая нулевой результат. Поэтому флаг Z устанавливается, и команда JZ 001Ah передаёт управление на адрес 001Ah. Команда по адресу 0018h не выполняется, что подтверждается неизменным значением регистра R5.



2.4.5 Блок операции М->R (чтение из RAM в РОН)

После декодирования команды автомат управления переходит в состояние S_CACHE_READ. На адресный вход кэша передаётся значение IR1, сигнал записи сбрасывается, а сигнал запроса cache_req_o устанавливается в активное состояние. Если требуемое слово находится в кэше, данные сразу выдаются процессору. Если происходит

промах, кэш обращается к ОЗУ данных через арбитр, получает слово из памяти, записывает его в выбранную строку кэша и выдаёт процессору.

После появления $cache_ready_i = 1$ автомат переходит в состояние $S_RF_WRITE_MEM$, где считанное слово записывается в выбранный регистр общего назначения. Затем команда завершается и автомат переходит к выборке следующей команды.

Команда $MOV\ 0020h, R1$ кодируется словами $IR0 = 0110h$ и $IR1 = 0020h$.

После её выполнения в регистре R1 появляется значение 8001h.

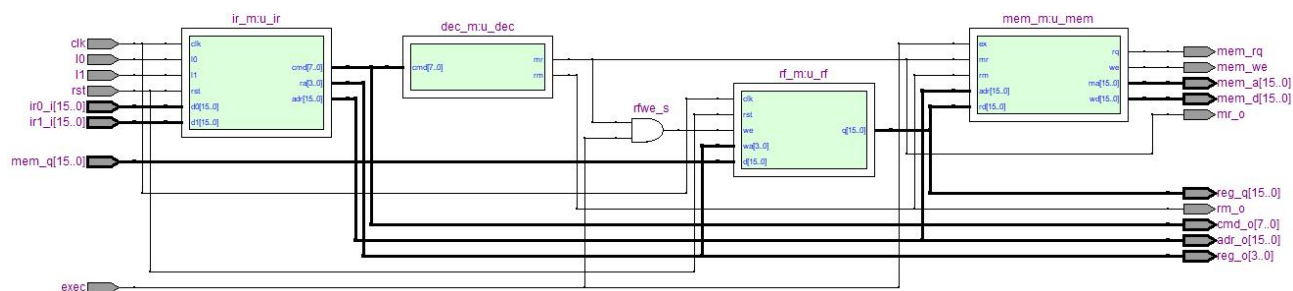


Рисунок 2.15 - Маршрутизация данных при выполнении операций M->R и R->M

2.4.6 Блок операции R->M (запись из РОН в RAM)

Команда $MOV\ Rn, adr$ имеет код операции 02h и выполняет запись содержимого выбранного регистра общего назначения в память данных. Номер регистра задаётся полем REG_NUM , а адрес записи задаётся вторым словом команды $IR1$.

После декодирования команды автомат управления переходит в состояние S_CACHE_WRITE . На адресный вход кэша передаётся значение $IR1$, на вход данных кэша передаётся содержимое выбранного регистра, а признак записи устанавливается в активное состояние.

Кэш-память в проекте работает в режиме сквозной записи. Поэтому при записи обновляется строка кэша и одновременно формируется обращение к ОЗУ данных. На схеме маршрутизации это подтверждается активным импульсом DBG_RAM_WE .

Команда $MOV\ R3, 0030h$ кодируется словами $IR0 = 0230h$ и $IR1 = 0030h$. К моменту её выполнения в регистре R3 находится значение C001h, поэтому это значение записывается в ячейку ОЗУ с адресом 0030h.

Большинство команд после выполнения переходят в состояние S_FINISH . В этом состоянии автомат завершает текущую операцию, увеличивает PC на два и возвращается к выборке следующей

команды. Увеличение на два связано с фиксированным двухсловным форматом команды.

Для команд JMP и JZ значение PC изменяется непосредственно в состояниях перехода. Для команды HLT возврат к выборке не выполняется: автомат остаётся в состоянии S_HALT.

Таким образом, блок выполнения общих операций задаёт единый алгоритм работы процессорного ядра: выборка двух слов команды, декодирование, выбор исполнительного маршрута, выполнение операции, запись результата и переход к следующей команде.

2.5 Арифметико-логические устройства

Арифметико-логическое устройство и блок регистров общего назначения образуют вычислительную часть процессорного ядра. Блок РОН хранит промежуточные данные и результаты выполнения команд, а АЛУ выполняет арифметико-логические преобразования над 16-разрядными словами. Результаты операций возвращаются в выбранный регистр общего назначения, а признаки результата фиксируются в регистре флагов.

Структурная схема арифметико-логического устройства приведена в приложении Ж.

В разработанной микро-ЭВМ АЛУ выполняет операции OR, NOR, SRA и INCS. Дополнительно в пакете команд предусмотрена внутренняя операция PASS_A, используемая как нейтральный режим передачи операнда без изменения. Пользовательскими командами являются OR Rn, adr, NOR Rn, adr, SRA Rn и INCS Rn.

Состав блока и назначение основных элементов схемы:

- блок регистров общего назначения U_RF - хранение операндов и результатов вычислений;
- арифметико-логическое устройство U_ALU - выполнение операций OR, NOR, SRA, INCS;
- входные регистры alu_a_r и alu_b_r - фиксация операндов, поступающих на АЛУ;
- регистр кода операции alu_op_r - выбор выполняемой операции АЛУ;
- выход результата alu_y - передача результата в тракт записи РОН;
- регистр флагов Z, S, C, O - хранение признаков результата;
- управляющий сигнал rf_we - разрешение записи результата в РОН;
- управляющий сигнал flag_we - разрешение обновления регистра флагов.

Назначение входных сигналов вычислительного блока:

- clk_i - тактовый сигнал, по которому выполняется запись в РОН и регистр флагов;
- rst_i - сброс регистров общего назначения и флагов;
- rf_a[15..0] - первый операнд, считанный из выбранного регистра РОН;
- rf_b[15..0] - второй операнд, считанный из РОН при необходимости;
- cache_rdata_i[15..0] - операнд, считанный из памяти данных через кэш для команд OR и NOR;
- flag_s - текущее значение признака знака, используемое в операции INCS;
- cur_reg_r[3..0] - номер регистра, участвующего в операции;
- cur_opcode_r[7..0] - код выполняемой команды.

Назначение выходных сигналов вычислительного блока:

- rf_wr_addr[3..0] - номер регистра, в который записывается результат;
- rf_din[15..0] - данные, записываемые в регистровый файл;
- alu_y[15..0] - результат операции АЛУ;
- alu_z, alu_s, alu_c, alu_o - признаки результата, сформированные АЛУ;
- dbg_r1_o-dbg_r7_o - отладочные выходы регистров;
- dbg_flags_o[3..0] - отладочный выход регистра флагов.

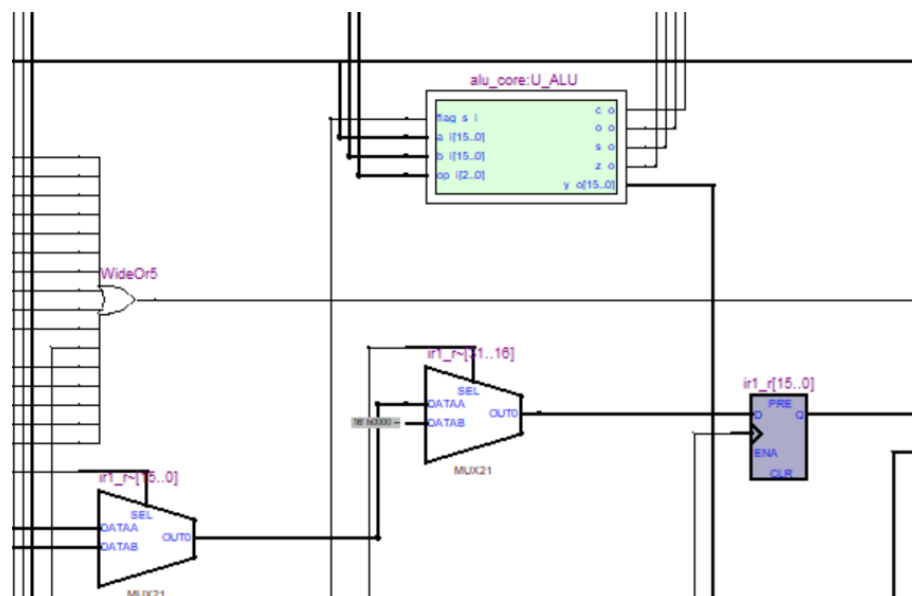


Рисунок 2.16 - Арифметико-логическое устройство

На схеме АЛУ входные операнды A[15..0] и B[15..0] поступают на комбинационную логику выбора операции. Код ALU_OP[2..0] определяет, какой результат будет передан на выход Y[15..0]. Параллельно формируются

признаки результата: Z устанавливается при $Y = 0000h$, S повторяет старший бит результата, C используется как бит переноса или вытесненный младший бит при сдвиге, O показывает переполнение при выполнении INCS.

2.5.1 Блоки арбитража к общим шинам

Первый операнд АЛУ поступает из выбранного регистра общего назначения через выход `rf_a`. Второй операнд зависит от команды. Для операций OR и NOR он считывается из памяти данных через кэш и поступает на вход АЛУ как `cache_rdata_i`. Для операций SRA и INCS обращение к памяти не требуется: используется содержимое регистра `Rn`, а для INCS дополнительно учитывается значение флага S. После формирования результата АЛУ автомат переходит в состояние `S_ALU_WRITE`. В этом состоянии результат `alu_u` записывается в выбранный регистр общего назначения, а признаки результата записываются в регистр флагов.

2.5.2 Блок регистров арифметико-логического устройства

Арифметико-логическое устройство реализовано как 16-разрядный комбинационный блок. На входы АЛУ поступают операнды `A[15..0]` и `B[15..0]`, код выполняемой операции `ALU_OP[2..0]` и текущее значение флага S, используемое командой INCS.

В состав АЛУ входят логический узел OR/NOR, схема арифметического сдвига вправо, сумматор для операции INCS и мультиплексор выбора результата. Результат формируется на выходе `Y[15..0]`. Параллельно вычисляются признаки результата: Z, S, C и O.

Назначение входных пинов АЛУ:

- `a_i[15..0]` - первый операнд;
- `b_i[15..0]` - второй операнд;
- `flag_s_i` - вход текущего значения флага S для операции INCS;
- `op_i[2..0]` - код выполняемой операции.

Назначение выходных пинов АЛУ:

- `y_o[15..0]` - результат операции;
- `z_o` - признак нулевого результата;
- `s_o` - признак знака результата;
- `c_o` - признак переноса или вытесненного младшего бита при сдвиге;
- `o_o` - признак переполнения.

Таблица 2.7 - Кодирование операций АЛУ

Код ALU_OP[2..0]	Операция	Назначение
000	PASS_A	передача операнда А без изменения
001	OR	поразрядное логическое ИЛИ
010	NOR	инверсия результата поразрядного ИЛИ
011	SRA	арифметический сдвиг вправо
100	INCS	инкремент при активном флаге S

2.5.3 Блок операции OR (побитовое «ИЛИ»)

Команда OR Rn, adr выполняет побитовое логическое ИЛИ между содержимым регистра Rn и словом памяти данных по адресу adr. Первый операнд поступает из регистрового файла, а второй считывается из памяти данных через кэш.

После декодирования команды процессор переходит в состояние S_CACHE_READ, получает слово памяти и передаёт его на второй вход АЛУ. Затем задаётся код операции ALU_OR. Результат вычисляется по выражению:

$$Y = A \text{ OR } B.$$

После выполнения операции результат записывается обратно в регистр Rn. Флаг Z устанавливается при нулевом результате, флаг S копирует старший бит результата. Флаги C и O для логической операции сбрасываются. Команда OR R2, 0022h показывает работу логического тракта АЛУ и обмена с памятью данных.

2.5.4 Блок операции NOR (побитовое «ИЛИ-НЕ»)

Команда NOR Rn, adr выполняет инверсию результата побитового ИЛИ регистра Rn и слова памяти данных. Как и для команды OR, первый операнд поступает из регистра общего назначения, а второй операнд считывается из памяти через кэш.

Результат операции формируется по выражению:

$$Y = \text{NOT}(A \text{ OR } B).$$

Полученное значение записывается обратно в выбранный регистр общего назначения. Флаги Z и S формируются по результату операции, а флаги C и O сбрасываются.

Команда NOR R4, 0025h может использоваться для формирования нулевого результата. После её выполнения устанавливается флаг Z, что затем проверяется командой условного перехода JZ.

2.5.5 Блок операции SRA (арифметический сдвиг вправо)

Команда SRA Rn выполняет арифметический сдвиг содержимого регистра Rn вправо на один разряд. Второе слово команды выбирается из ПЗУ в рамках общего формата, но при выполнении не используется.

При арифметическом сдвиге старший бит исходного операнда сохраняется, поэтому знак числа не изменяется. Остальные биты сдвигаются вправо, а младший вытесненный бит записывается во флаг C.

Для операнда A результат формируется следующим образом:

$$Y(15) = A(15);$$

$$Y(i) = A(i+1), \text{ где } i = 0 \dots 14.$$

После команды MOV 0020h, R1 в регистре R1 находится значение 8001h. После выполнения SRA R1 оно преобразуется в C000h, а вытесненный младший бит записывается во флаг C.

2.5.6 Блок операции INCS (инкремент по флагу S)

Команда INCS Rn выполняет инкремент выбранного регистра на значение флага S. В отличие от обычного инкремента, ко входу сумматора подаётся не постоянная единица, а текущее значение флага знака.

Результат операции описывается выражением:

$$Y = A + S.$$

Если $S = 0$, содержимое регистра не изменяется. Если $S = 1$, значение регистра увеличивается на единицу. После вычисления результат записывается обратно в выбранный регистр, а флаги Z, S, C и O обновляются по результату операции.

Команда INCS R1 может выполняться после SRA R1. Так как после сдвига значение R1 = C000h имеет установленный старший бит, флаг S равен единице. Поэтому команда INCS R1 изменяет значение регистра на C001h.

2.5.7 Блок формирования регистра флагов

Регистр флагов фиксирует признаки результата арифметико-логических операций. В проекте используются четыре флага:

- Z - признак нулевого результата;
- S - признак знака результата;
- C - перенос или вытесненный бит;
- O - признак переполнения.

Флаг Z устанавливается, если результат АЛУ равен 0000h. Флаг S повторяет старший бит результата.

Флаг C при операции SRA получает младший вытесненный бит исходного операнда, а при INCS используется как перенос из старшего разряда. Флаг O используется для фиксации переполнения при выполнении INCS; для логических операций и сдвига он сбрасывается.

Таблица 2.8 - Реализация операций АЛУ и обновление флагов

Операция	Аппаратное выражение	Запись результата	Обновление флагов
OR Rn, adr	$Y = A \text{ OR } B$	$Rn \leftarrow Y$	$Z = (Y = 0), S = Y_{15}, C = 0, O = 0$
NOR Rn, adr	$Y = \text{NOT}(A \text{ OR } B)$	$Rn \leftarrow Y$	$Z = (Y = 0), S = Y_{15}, C = 0, O = 0$
SRA Rn	$Y_{15} = A_{15}, Y_i = A_{(i+1)}$	$Rn \leftarrow Y$	$Z = (Y = 0), S = Y_{15}, C = A_0, O = 0$
INCS Rn	$Y = A + S$	$Rn \leftarrow Y$	$Z = (Y = 0), S = Y_{15}, C - \text{перенос}, O - \text{переполнение}$

Операция SRA используется для арифметического сдвига вправо с сохранением знака. Например, при A = 8001h результат равен C000h, а во флаг C заносится вытесненный младший бит. Операция INCS используется для прибавления единицы только при активном флаге S. Поэтому при A = C000h

и $S = 1$ результат равен C001h. Операции OR и NOR выполняются над двумя 16-разрядными операндами.

Для операции SRA сохраняется старший бит исходного операнда, а младший бит заносится во флаг C. Для OR и NOR используются оба операнда, при этом C и O сбрасываются. Для INCS к операнду прибавляется текущее значение флага S, поэтому при $S = 0$ регистр не изменяется, а при $S = 1$ увеличивается на единицу.

Таким образом, арифметико-логическое устройство и блок РОН обеспечивают выполнение всех вычислительных команд проектируемой микро-ЭВМ.

Команды OR, NOR, SRA и INCS проверяют логическую обработку, арифметический сдвиг, условный инкремент и корректное формирование флагов.

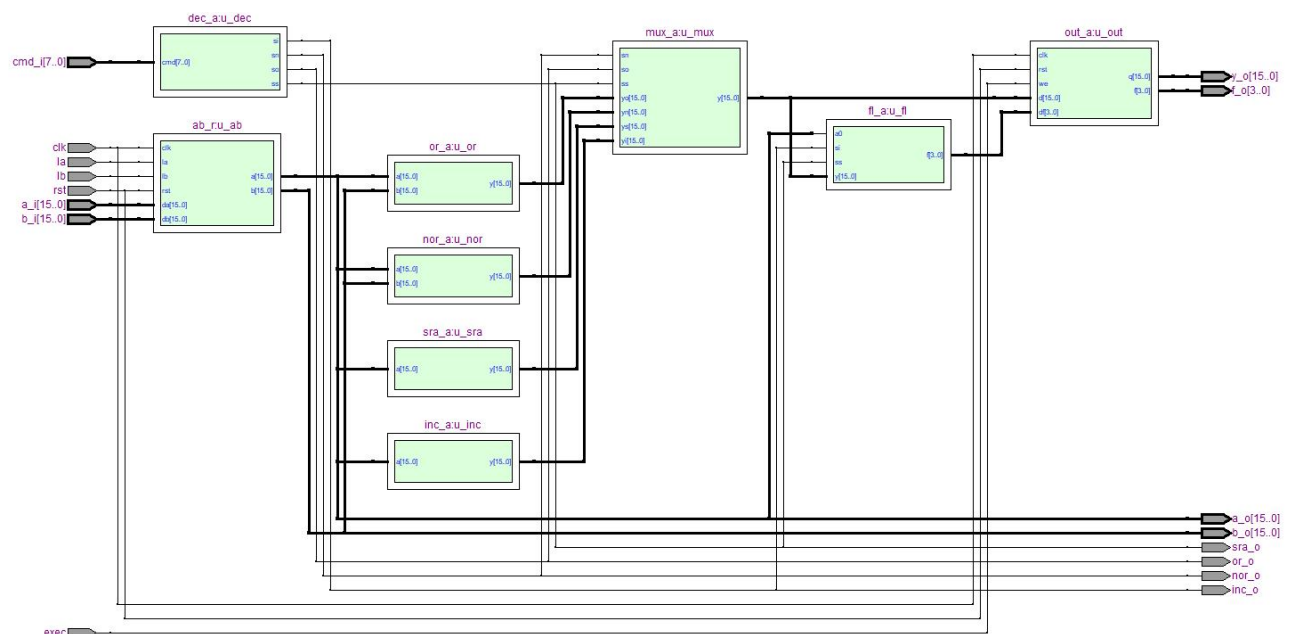


Рисунок 2.17 - Маршрутизация операндов при выполнении операций АЛУ

2.6 Стековое запоминающее устройство

Стековое запоминающее устройство предназначено для временного хранения 16-разрядных слов при выполнении команд PUSH и POP. В отличие от ОЗУ данных, стек не адресуется полем адреса команды: выбор ячейки выполняется автоматически по указателю SP. Поэтому стек используется как локальное быстрое хранилище процессорного ядра и не занимает общую шину памяти при выполнении стековых операций. Структурная схема стекового устройства приведена в приложении Д.

В проектируемой микро-ЭВМ стек имеет глубину 7 слов и работает по принципу LIFO: слово, записанное последним, считывается первым. Такая организация удобна для сохранения промежуточных результатов, передачи значения между командами и восстановления ранее записанного регистра без явного указания адреса ячейки.

Стек растёт вниз. Указатель SP имеет разрядность 3 бита и хранит номер текущей вершины стека. При пустом стеке используется специальное состояние $SP = 7$, так как реальные ячейки имеют номера от 0 до 6. После первой операции PUSH слово записывается в ячейку 6, а SP принимает значение 6. При следующих занесениях указатель последовательно уменьшается до 0; состояние $SP = 0$ соответствует полностью занятому стеку.

2.6.1 Блоки арбитража к общим шинам

Стековый блок подключается к процессорному ядру через внутренний тракт регистров общего назначения и не обращается напрямую к ОЗУ данных. За счёт этого операции PUSH и POP не конкурируют с кэшем, КПДП и арбитром внешней шины данных. Доступ к стеку разрешается только устройством управления, которое формирует взаимоисключающие сигналы `stack_push` и `stack_pop`.

При выполнении PUSH источником данных является выбранный регистр общего назначения. Его содержимое поступает на вход стека через регистровый тракт, после чего активируется сигнал записи. Остальные исполнительные блоки при этом не передают данные на вход стека, поэтому конфликт источников исключается.

При выполнении POP направление обмена меняется: стек выдаёт слово с вершины на внутреннюю линию данных, а устройство управления разрешает запись этого слова в выбранный РОН. Запись в регистр выполняется в отдельной фазе, после завершения чтения из стека. Такое разделение фаз позволяет не совмещать чтение вершины и запись результата в РОН в одном состоянии автомата.

Сигналы `empty` и `full` служат для контроля граничных состояний. Признак `empty` активен при $SP = 7$ и показывает, что в стеке нет сохранённых слов. Признак `full` активен при $SP = 0$ и показывает, что заняты все семь ячеек. Эти признаки позволяют устройству управления не изменять содержимое стека при недопустимых обращениях.

2.6.2 Блок схемы стека

Исполнительная часть стека представляет собой регистровый массив из семи 16-разрядных ячеек, указатель SP и выходной регистр данных. Все изменения выполняются синхронно по фронту тактового сигнала. При сбросе содержимое ячеек очищается, выходное слово становится равным 0000h, а SP устанавливается в значение 7, соответствующее пустому стеку.

Алгоритм PUSH состоит из двух действий. Сначала проверяется, что стек не полон, то есть SP больше 0. Затем входное слово записывается в ячейку с номером SP - 1, а указатель уменьшается на единицу. Например, если перед занесением SP = 7, то запись выполняется в ячейку 6 и после такта SP = 6; если перед занесением SP = 4, то слово помещается в ячейку 3 и SP становится равным 3.

Алгоритм POP выполняется в обратном порядке. Если стек не пуст, то сначала считывается ячейка, на которую указывает SP, а затем указатель увеличивается на единицу. При SP = 6 считывается ячейка 6, после чего стек снова переходит в пустое состояние SP = 7. При наличии нескольких записанных слов это обеспечивает чтение в порядке, обратном порядку занесения.

Для команды PUSH Rn используется код операции 07h. После декодирования команды устройство управления выбирает номер регистра Rn, передаёт его содержимое на вход стека и переводит автомат в фазу S_STACK_PUSH. В этой фазе активируется запись в стек, а после завершения операции выполняется переход к выборке следующей команды.

Для команды POP Rn используется код операции 08h. После декодирования команды устройство управления переводит автомат в фазу S_STACK_POP, где считывается вершина стека. Затем выполняется фаза S_RF_WRITE_POP, в которой считанное слово записывается в регистр Rn. Разделение на две фазы нужно потому, что значение с выхода стека должно быть зафиксировано до записи в РОН.

Работу стека можно проследить по указателю DBG_SP. После занесения одного слова указатель изменяется с 7 на 6, а после извлечения этого слова возвращается с 6 на 7. Если последовательно выполнить несколько команд PUSH, то указатель будет уменьшаться, а команды POP будут возвращать его к состоянию пустого стека. Поэтому изменение DBG_SP однозначно показывает текущее заполнение стекового устройства.

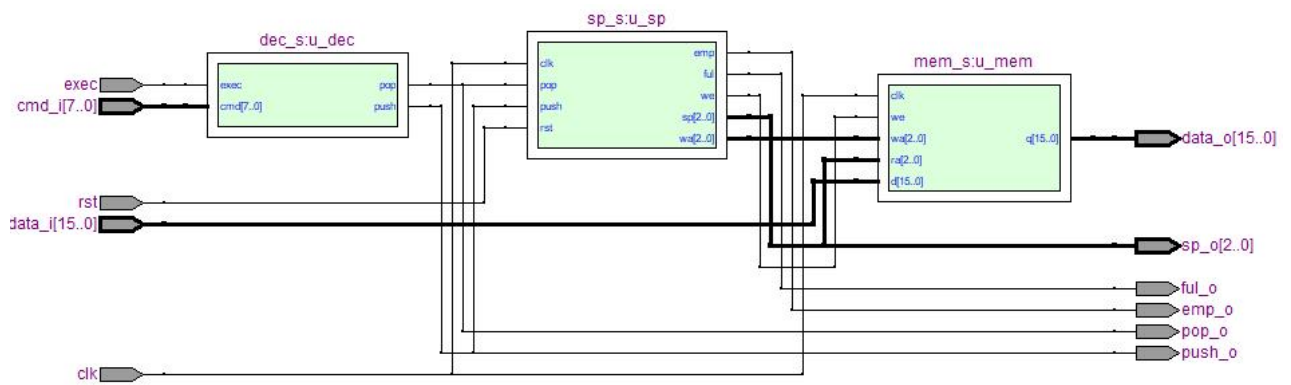


Рисунок 2.18 - Изменение указателя стека при выполнении PUSH и POP

2.7 КЭШ-память

2.7.1 Блок данных КЭШа

Кэш-память располагается между исполнительской частью процессора и ОЗУ данных и предназначена для уменьшения среднего времени доступа к данным. По техническому заданию для разрабатываемой микро-ЭВМ требуется кэш с параметром $k = 4$, алгоритмом замещения по наибольшей давности хранения и синхронизацией с памятью по схеме «сквозная запись с отображением». Для дальнейшей разработки принимается четырёхканальный множественно-ассоциативный кэш с 16 наборами и длиной строки в одно 16-разрядное слово. Такое решение упрощает схему сравнения тегов, не требует выделения поля смещения внутри строки и хорошо подходит для учебного моделирования.

Структурная схема кэш-памяти приведена в приложении И.

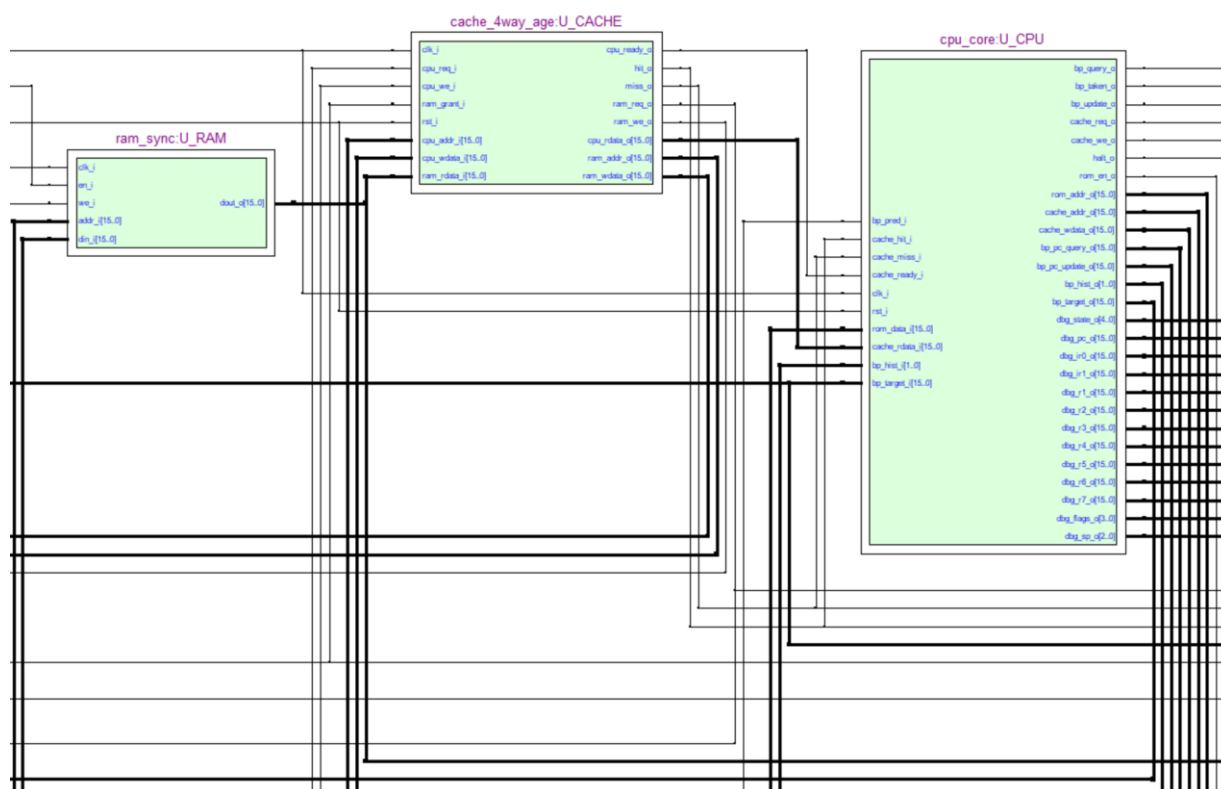


Рисунок 2.19 - Расположение блока кэш-памяти в составе подсистемы памяти.

При 16 наборах адрес данных $A[15:0]$ делится на две части: младшие четыре бита $A[3:0]$ образуют индекс набора, а старшие двенадцать бит $A[15:4]$ записываются в поле тега. В каждом наборе имеются четыре строки, каждая из которых содержит слово данных, тег, признак действительности и поле возраста. Поскольку используется сквозная запись, хранить признак модификации *dirty* не требуется: любое изменение слова немедленно дублируется в ОЗУ.

Таблица 9 - Состав строки кэша данных

Поле строки	Разрядность	Назначение
V	1 бит	Признак того, что строка содержит корректные данные и может участвовать в сравнении тегов.
TAG	12 бит	Старшие биты адреса $A[15:4]$, используемые для определения попадания в выбранном наборе.

DATA	16 бит	Кэшируемое слово данных, выдаваемое процессору при попадании.
AGE	2 бита	Код относительной давности хранения в пределах набора; используется при выборе строки на замещение.

Попадание в кэш определяется следующим образом. По индексу $A[3:0]$ выбирается нужный набор, после чего тег адреса одновременно сравнивается с четырьмя тегами всех строк набора. Если хотя бы в одной строке $V = 1$ и TAG совпадает с $A[15:4]$, формируется сигнал hit, а на выход кэша через мультиплексор поступает соответствующее поле DATA. При отсутствии совпадений формируется miss и контроллер кэша запрашивает доступ к ОЗУ через арбитр шины.

Алгоритм замещения по наибольшей давности хранения реализуется проще, чем алгоритм замещения по наибольшей давности хранения, поскольку возраст строки меняется только в момент загрузки новой строки, а не при каждом чтении. Если в наборе имеется хотя бы одна недействительная строка, новая запись помещается в неё. Если набор полностью заполнен, выбирается строка с максимальным значением AGE. После загрузки новой строки её возраст устанавливается равным нулю, а возраста остальных действительных строк набора увеличиваются на единицу с насыщением до максимума. При попадании значения AGE не меняются, так как критерий замещения связан не с последним обращением, а именно с длительностью хранения строки в кэше.

Политика «сквозная запись с отображением» трактуется следующим образом. При записи по адресу, который уже присутствует в кэше, слово одновременно обновляется и в строке кэша, и в ОЗУ. При записи по адресу, отсутствующему в кэше, слово сначала передаётся в ОЗУ, после чего соответствующая строка создаётся и в кэше. Тем самым после любой операции записи кэш не расходится с основной памятью, а последующие чтения недавно записанного слова могут выполняться с попаданием.

С точки зрения временных диаграмм чтение при попадании укладывается в один цикл обращения к кэшу: в текущем такте выставляется адрес, а к концу такта или в следующем синхронном такте на выходе формируется слово данных. При промахе добавляются такты арбитража и чтения ОЗУ, после чего загруженная строка сразу же выдаётся в процессор. Для записи время операции в основном определяется синхронным ОЗУ,

потому что даже при попадании в кэш должно быть выполнено дублирование данных в оперативную память.

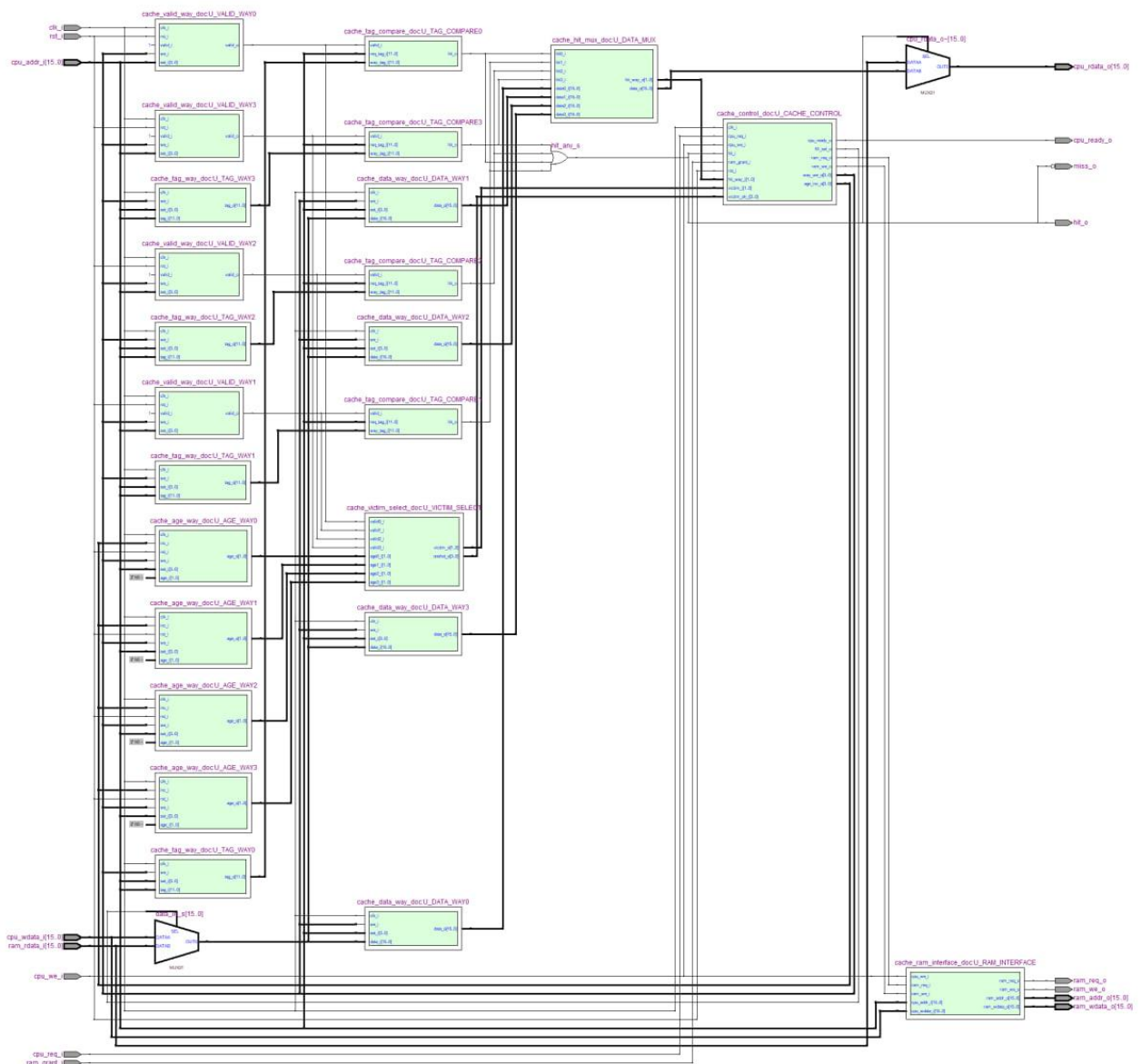


Рисунок 2.20 - Структурная схема блока кэш-памяти.

2.7.2 Второстепенные блоки КЭШа

Адрес обращения к кэш-памяти имеет разрядность 16 бит. Так как строка кэша содержит одно 16-разрядное слово, поле смещения внутри строки отсутствует. Младшие четыре разряда адреса используются как индекс набора, а старшие двенадцать разрядов образуют тег.

Таблица 2.9 - Разбиение адреса обращения к кэш-памяти

Разряды	Поле	Размер	Назначение
A[15:4]	TAG	12 бит	Сравнивается с тегами всех way выбранного набора
A[3:0]	SET	4 бита	Выбирает один из 16 наборов кэш-памяти
-	OFFSET	0 бит	Смещение отсутствует, строка кэша содержит одно 16-разрядное слово

Поле SET=A[3:0] выбирает один из 16 наборов кэш-памяти. В выбранном наборе параллельно проверяются четыре way. Поле TAG=A[15:4] сравнивается с тегами, сохраненными в каждом way, а результат сравнения совместно с valid-битом определяет попадание или промах.

2.7.3 Блок управления КЭШа

На рисунке 2.21 показан фрагмент структурной схемы автомата управления кэш-памятью. Автомат state_r формирует состояния IDLE, WAIT_READ_GRANT, WAIT_READ_DATA и WAIT_WRITE_GRANT. В состоянии IDLE выполняется проверка обращения процессора к кэшу, при промахе чтения выполняется переход к ожиданию доступа к ОЗУ, а при записи - к состоянию сквозной записи.

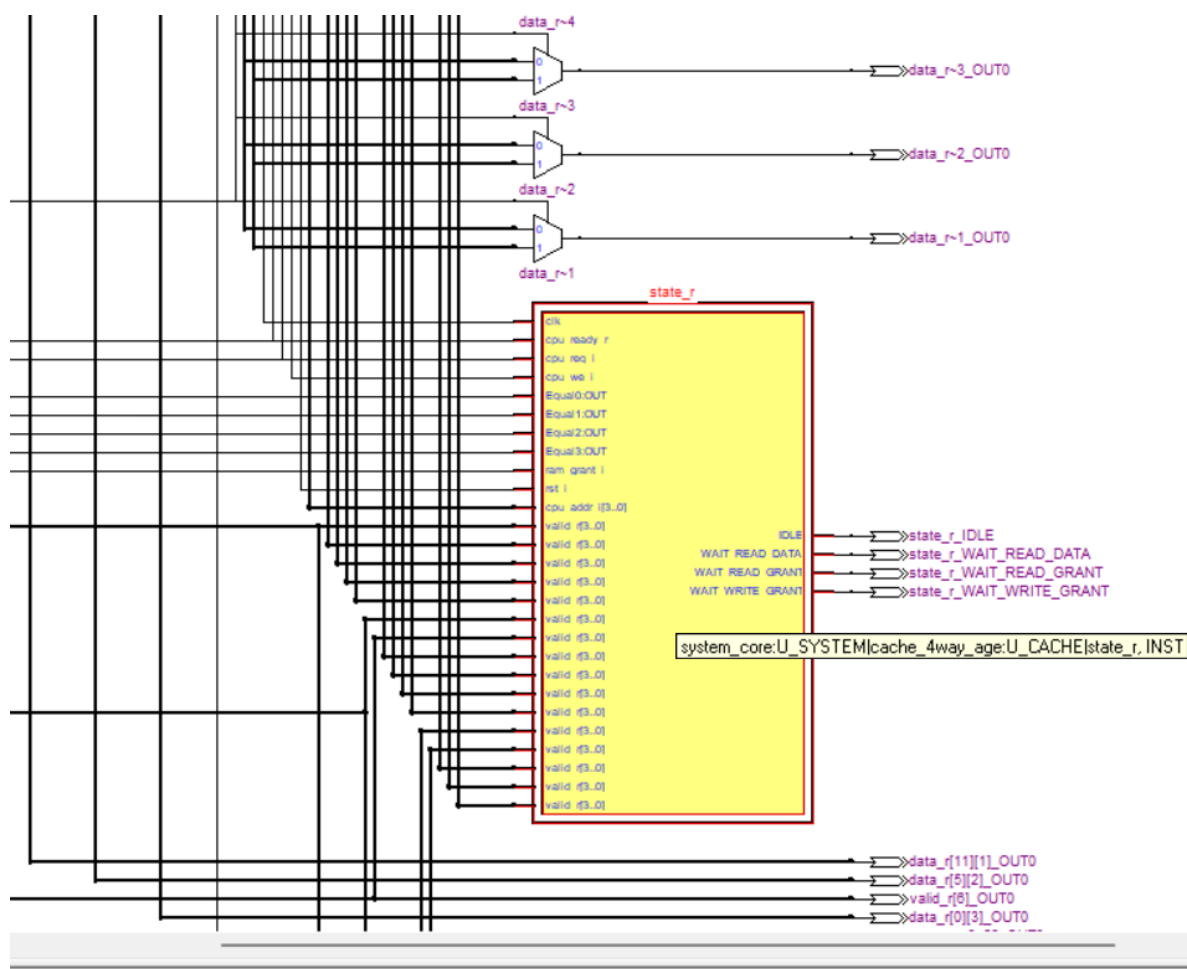


Рисунок 2.21 - Автомат управления кэш-памятью

Для проверки работы кэш-памяти выполнено функциональное моделирование отдельного блока `cache_4way_age`. На временной диаграмме показаны три характерных режима: чтение при промахе, повторное чтение при попадании и запись в режиме сквозной записи.

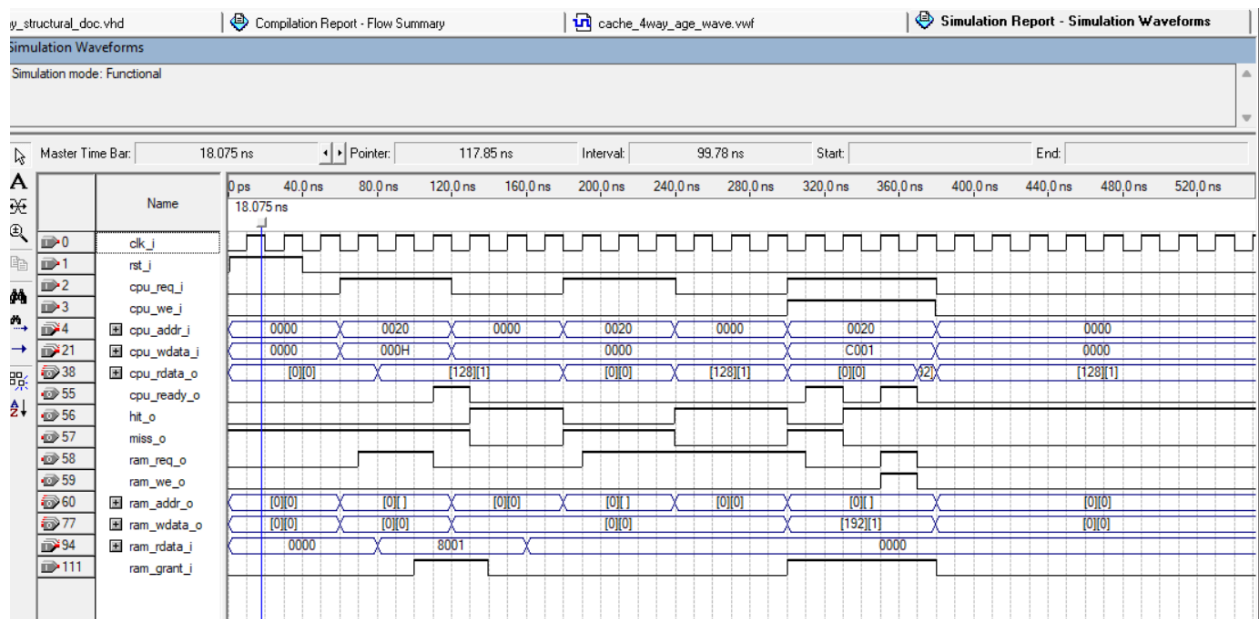


Рисунок 2.22 - Временная диаграмма функционального моделирования кэш-памяти

При первом обращении к адресу 0020h кэш пуст, поэтому формируется сигнал miss_o, выставляется запрос ram_req_o, а адрес передается на ram_addr_o. После разрешения ram_grant_i слово 8001h считывается из ОЗУ, записывается в выбранную строку кэша и выдается процессору через cpu_rdata_o. При повторном обращении к адресу 0020h совпадают тег и valid-бит, поэтому формируется hit_o, а обращение к ОЗУ не выполняется. При операции записи кэш обновляет свою строку и одновременно выставляет ram_we_o, ram_addr_o и ram_wdata_o, что подтверждает режим сквозной записи.

Работа кэш-памяти проверялась также в составе микро-ЭВМ. Для наблюдения использованы отладочные выходы верхнего уровня, отображающие состояние процессора, обращения к кэш-памяти, сигналы арбитража и обмен с ОЗУ.

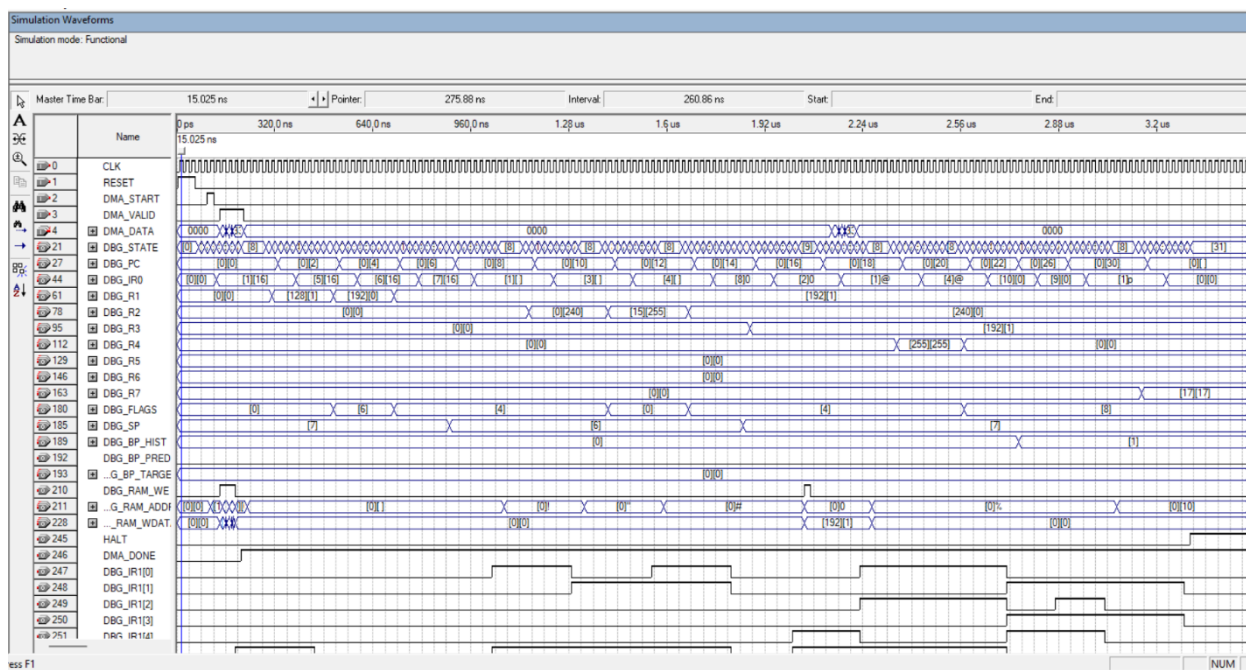


Рисунок 2.23 - Работа кэш-памяти при обращениях к памяти, часть 1

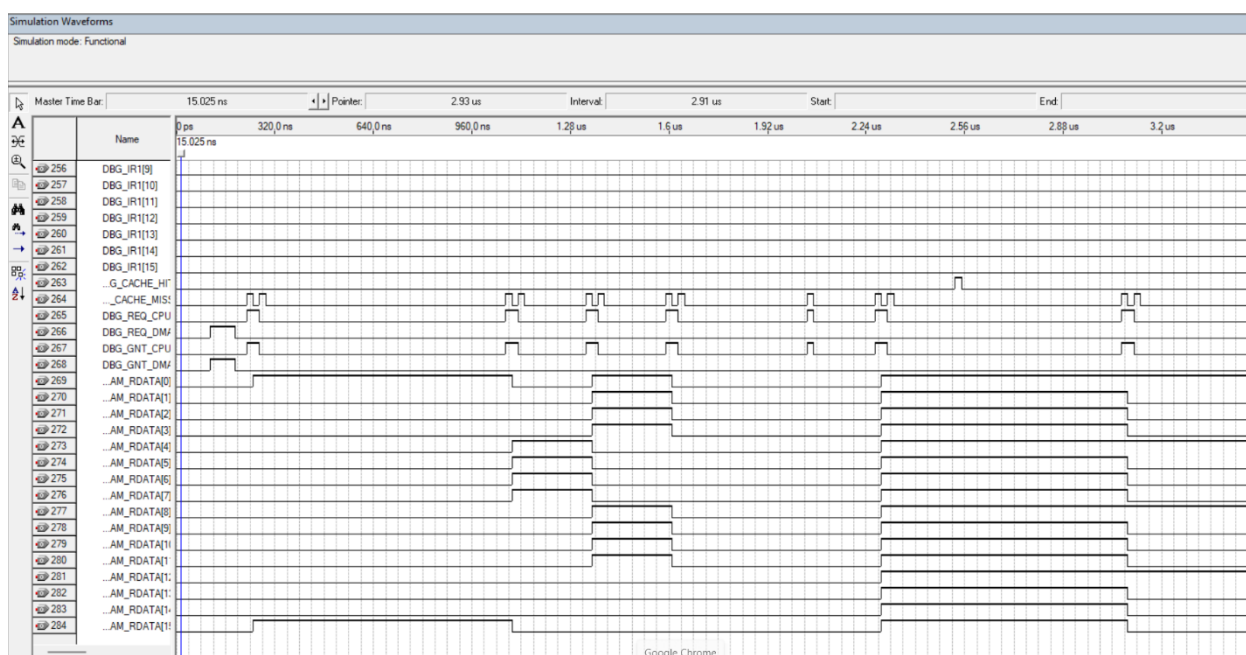


Рисунок 2.24 - Работа кэш-памяти при обращениях к памяти, часть 2

На временной диаграмме видно, что при обращениях процессора к памяти формируются сигналы `DBG_CACHE_HIT` и `DBG_CACHE_MISS`. При промахе кэш запрашивает доступ к ОЗУ через `DBG_REQ_CPU`, после чего арбитр выдает разрешение `DBG_GNT_CPU`. Адрес и данные обмена отображаются сигналами `DBG_RAM_ADDR`, `DBG_RAM_WDATA` и `DBG_RAM_RDATA`.

При записи активируется DBG_RAM_WE, что подтверждает выполнение сквозной записи в ОЗУ.

Таким образом, временное моделирование подтверждает работу основных режимов кэш-памяти: промах чтения с загрузкой строки из ОЗУ, попадание при повторном чтении и сквозную запись в ОЗУ. Использование полей TAG и SET обеспечивает выбор набора и проверку попадания, а счетчики AGE применяются для выбора строки при замещении.

2.8 Контроллер прямого доступа к памяти

Контроллер прямого доступа к памяти предназначен для обмена данными между внешним устройством и ОЗУ без участия арифметико-логического тракта процессора. Для технического задания заданы начальный адрес 10 и объём блока 6 байт. Поскольку в проекте принята словная организация ОЗУ по 16 бит, это соответствует передаче трёх последовательных слов по адресам 10, 11 и 12. Такое фиксированное задание удобно для моделирования, потому что позволяет заранее задать регистры адреса и счётчика и затем наблюдать корректность заполнения памяти по дампу. Структурная схема контроллера прямого доступа к памяти приведена в приложении Л.

В состав КПДП входят регистр начального адреса START_ADDR, рабочий регистр текущего адреса CURR_ADDR, счётчик оставшегося объёма BYTE_CNT, входной регистр данных DATA_BUF, регистр состояния STATUS и конечный автомат управления. Перед началом передачи в START_ADDR загружается значение 10, а в BYTE_CNT - значение 6. После поступления внешнего запроса контроллер копирует начальный адрес в CURR_ADDR, переводит STATUS в состояние BUSY и запрашивает у арбитра доступ к системной шине данных.

Таблица 11 - Регистры и сигналы контроллера прямого доступа к памяти

Элемент	Разрядность/начальное значение	Назначение
START_ADDR	16 бит / 10	Хранит базовый адрес области ОЗУ, в которую должен быть записан принимаемый блок.
CURR_ADDR	16 бит	Содержит текущий адрес очередной передачи;

		увеличивается после каждого успешно записанного слова.
BYTE_CNT	16 бит / 6	Содержит число ещё не переданных байтов; после каждой 16-битной передачи уменьшается на 2.
DATA_BUF	16 бит	Буферизует слово, принимаемое от внешнего устройства до момента записи в ОЗУ.
STATUS	несколько бит	Формирует признаки BUSY, DONE и, при необходимости, ERROR.
DREQ, DACK, BR_DMA, BG_DMA	служебные сигналы	Организируют внешний запрос на передачу и захват системной шины через арбитр.

В данной работе КПП рассматривается в режиме передачи от внешнего источника в ОЗУ. Алгоритм его работы таков. Сначала внешнее устройство формирует сигнал DREQ и выдаёт первое 16-разрядное слово на вход DATA_BUF. Контроллер выставляет запрос BR_DMA на арбитр шины. После получения разрешения BG_DMA он подаёт адрес CURR_ADDR на вход ОЗУ, выдаёт данные из DATA_BUF на шину DI_RAM и формирует сигнал записи WR_RAM. После завершения такта записи CURR_ADDR увеличивается на единицу, а BYTE_CNT уменьшается на 2. Цикл повторяется до тех пор, пока счётчик не достигнет нуля.

Так как объём блока составляет всего 6 байт, для упрощения управляющей логики выбирается пакетный режим: после получения гранта контроллер удерживает шину до завершения всех трёх слов передачи. По окончании обмена признак BUSY сбрасывается, формируется DONE, а арбитру передаётся освобождение шины. Если требуется прерывание процессора по завершении обмена, сигнал DONE может быть дополнительно подан на вход контроллера прерываний или на отдельный флаг состояния. Использование КПП особенно удобно при моделировании, поскольку позволяет показать, что память данных изменяется даже без выполнения процессором явных команд MOV reg, adr. В дампе памяти после окончания

передачи должны измениться ячейки с адресами 10, 11 и 12, а содержимое регистров процессора при этом может остаться неизменным. Именно эта особенность и демонстрирует принцип прямого доступа к памяти как самостоятельного шино-мастеринга со стороны внешнего контроллера.

2.9 Предсказатель переходов

Для технического задания задана схема предсказания переходов A4 с шаблоном «4, PC(2) || GHR(2)». В пояснительной записке целесообразно реализовать её как двухуровневый адаптивный предсказатель, использующий два младших бита адреса условного перехода и двухразрядный регистр глобальной истории. Такая структура хорошо согласуется с параметрами задания и при этом достаточно проста для моделирования на ПЛИС. Структурная схема предсказателя переходов приведена в приложении К [3].

В состав блока предсказания включаются: регистр глобальной истории GHR[1:0], таблица историй шаблонов PHT на 16 ячеек, а также буфер целевых адресов ВТВ на 16 записей. Индекс таблицы образуется конкатенацией двух младших бит адреса команды перехода и текущего содержимого GHR: INDEX = PC[1:0] || GHR[1:0]. По этому индексу выбирается 2-битный счётчик насыщения, задающий прогноз направления перехода.

Таблица 10 - Интерпретация 2-битного счётчика предсказания

Код	Состояние	Выдаваемый прогноз
00	сильно не переход	не переходить
01	слабо не переход	не переходить
10	слабо переход	переходить
11	сильно переход	переходить

В проектируемой системе условным переходом считается команда JZ adr, введённая как дополнение к обязательному набору команд. Для неё предсказатель решает две задачи: заранее выбрать предполагаемое направление и, в случае прогноза «переход», выдать адрес цели. Поскольку адрес перехода хранится во втором слове команды, для ранней подстановки нового значения IP в блоке предсказания дополнительно используется ВТВ. В каждой записи ВТВ хранятся признак действительности, тег адреса ветвления и ранее подтверждённый целевой адрес.

Во время выборки команды блок предсказания получает текущее значение IP. Если ВТВ содержит запись для данного адреса ветвления, а соответствующий счётчик РНТ находится в состоянии 10 или 11, формируется прогноз «переход» и в IP подставляется целевой адрес из ВТВ. В противном случае выбирается последовательное продолжение программы, то есть IP + 2, так как длина команды принята равной двум словам. Для безусловной команды JMP adr необходимость в предсказании направления отсутствует: она всегда считается выполняющейся с переходом, и её целевой адрес просто загружается в IP после декодирования.

После фактического выполнения условного перехода устройство управления возвращает в блок предсказания сигнал ACTUAL_TAKEN. По нему выбранный счётчик в РНТ увеличивается или уменьшается на единицу до ближайшего предельного состояния, а регистр глобальной истории обновляется по правилу $GHR_next = \{GHR[0], ACTUAL_TAKEN\}$. Если предсказание оказалось неверным, устройство управления сбрасывает ошибочно выбранную следующую команду, корректирует значение IP и обновляет ВТВ новым адресом цели. В результате на часто повторяющихся ветвлениях предсказатель постепенно подстраивается к характерному шаблону программы и уменьшает число пустых циклов на переходах.

2.10 Организация арбитража

Поскольку архитектура микро-ЭВМ является Гарвардской, выборка команд из ПЗУ выполняется по выделенному тракту и не требует участия общего арбитра данных. Следовательно, арбитраж необходим только для обращений к ОЗУ данных. Основными претендентами на использование этой шины в проекте являются процессорная подсистема данных, представленная контроллером кэша, и контроллер прямого доступа к памяти. По условию задания должен быть реализован централизованный параллельный арбитраж, то есть все запросы одновременно поступают в один управляющий узел, который формирует взаимоисключающие сигналы разрешения. Структурная схема арбитража шин приведена в приложении М.

Для данной микро-ЭВМ достаточно двух входов запроса: REQ_CPU и REQ_DMA. Сигнал REQ_CPU формируется контроллером кэша при промахе чтения или при сквозной записи в ОЗУ. Сигнал REQ_DMA выдаётся КППД на время пакетной передачи. Арбитр реализуется как комбинационный приоритетный блок с регистрацией результата в такте синхронизации. В принятом проектном решении более высокий приоритет получает REQ_DMA,

поскольку источник данных может быть внешним устройством реального времени, а объём обмена всего 6 байт, поэтому задержка центрального процессора будет небольшой.

Таблица 12 - Логика централизованного параллельного арбитра для шины данных

REQ_DMA	REQ_CPU	GNT_DMA	GNT_CPU
0	0	0	0
0	1	0	1
1	0	1	0
1	1	1	0

Из таблицы видно, что при одновременном поступлении двух запросов приоритет получает КППД. Это означает, что процессорная подсистема должна удерживать свой запрос до освобождения шины. После завершения пакетной передачи DMA арбитр снимает GNT_DMA и в следующем такте может выдать разрешение процессору. Благодаря выделенному тракту команд даже во время активного КППД содержимое указателя команд IP и выборка команд из ПЗУ не обязательно останавливаются, однако фактические обращения к ОЗУ данных со стороны процессора задерживаются до освобождения шины.

Централизованный параллельный арбитр удобен тем, что его схема легко описывается логическими формулами. Для выбранной двухмастерной конфигурации они имеют вид: $GNT_DMA = REQ_DMA$, $GNT_CPU = REQ_CPU \text{ and not } REQ_DMA$. При необходимости расширения системы к арбитру можно добавить новые входы запросов и перераспределить приоритеты, не меняя принципа работы уже существующих блоков. Это делает принятое решение удобной основой для последующего развития архитектуры.

Таким образом, во втором разделе получена уже не только общая, но и структурно детализированная архитектура микро-ЭВМ. Для памяти, АЛУ, устройства управления, кэша, предсказателя переходов, КППД и арбитра шины сформулированы конкретные проектные решения, которые далее могут быть непосредственно перенесены в структурные схемы и использованы при подготовке временных диаграмм и моделей проверки.

3 ФУНКЦИОНАЛЬНОЕ МОДЕЛИРОВАНИЕ

Функциональное моделирование проводится для проверки того, что разработанная структура микро-ЭВМ не только формально содержит требуемые блоки, но и правильно выполняет программу во времени. Во втором разделе рассмотрены состав и структурные схемы узлов, а в данном разделе анализируются временные диаграммы: выборка двухсловной команды, декодирование, обращение к памяти, выполнение операций АЛУ, работа стека, переходы, обмен через КПП и арбитраж общей шины данных [6].

В качестве контрольной используется программа, размещённая в ПЗУ. Каждая команда занимает два 16-разрядных слова: первое слово IR0 содержит код операции и номер регистра, второе слово IR1 содержит адрес операнда, адрес перехода или нулевое служебное слово. Такая организация делает цикл выборки одинаковым для всех команд и упрощает проверку временных диаграмм [9].

Для чтения диаграмм далее используются следующие обозначения тактов: T0 соответствует выдаче адреса первого слова команды, T1 - ожиданию и загрузке IR0, T2 - выдаче адреса второго слова, T3 - ожиданию и загрузке IR1, T4 - декодированию команды. Последующие такты зависят от типа команды: для обращений к памяти добавляется ожидание кэша и ОЗУ, для операций АЛУ - такт записи результата, для стековых команд - такты работы со стеком, для переходов - загрузка нового значения указателя команд.

3.1 Функциональное моделирование блока памяти

Результаты функционального моделирования блока памяти показаны на рисунке 3.1. Проверка охватывает два направления обмена: чтение слов программы из ПЗУ и обращение к ОЗУ данных. ПЗУ используется при выборке команд по значению указателя команд РС, а ОЗУ используется при командах MOV, OR, NOR и при обмене через контроллер прямого доступа к памяти.

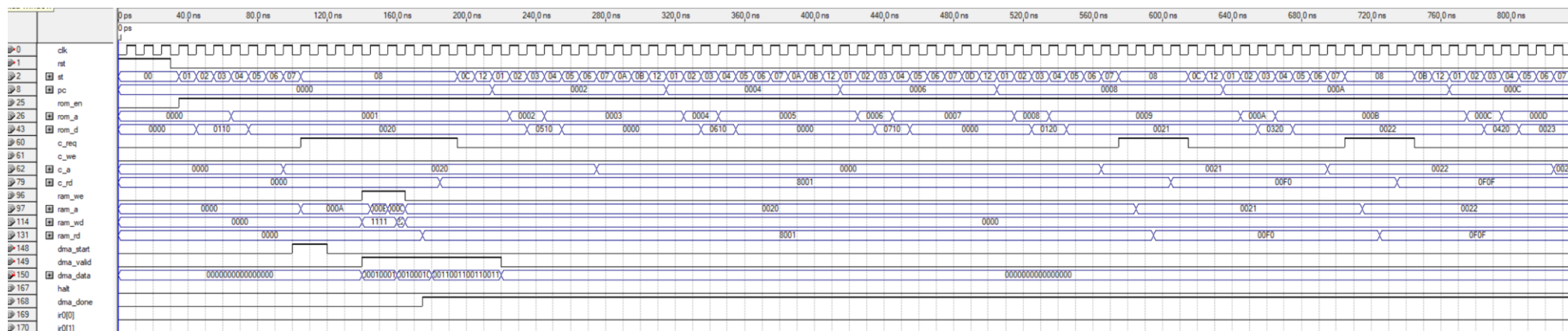


Рисунок 3.1 - Результаты функционального моделирования блока памяти

T0. На адресный вход ПЗУ подаётся PC = 0000h, активируется разрешение чтения. На выходе ПЗУ появляется первое слово команды IR0 = 0110h, соответствующее операции MOV 0020h, R1.

T1. Адрес ПЗУ увеличивается до 0001h, считывается второе слово IR1 = 0020h. Оно используется как адрес операнда в ОЗУ данных. После завершения выборки команда может быть декодирована и направлена на чтение памяти данных.

T2. Для команды MOV adr, Rn на адресную шину ОЗУ подаётся значение 0020h. Сигнал записи RAM_WE остаётся равным нулю, поэтому выполняется чтение. Позже при команде MOV R3, adr на адрес 0030h подаётся записываемое значение C001h и активируется RAM_WE. Тем самым диаграмма подтверждает оба режима работы ОЗУ: чтение и запись.

3.2 Функциональное моделирование декодера команд

Результаты функционального моделирования декодера команд показаны на рисунке 3.2. Декодер анализирует старший байт первого слова команды IR0 и формирует признаки операции. Младшие разряды IR0 используются как номер регистра, участвующего в операции.

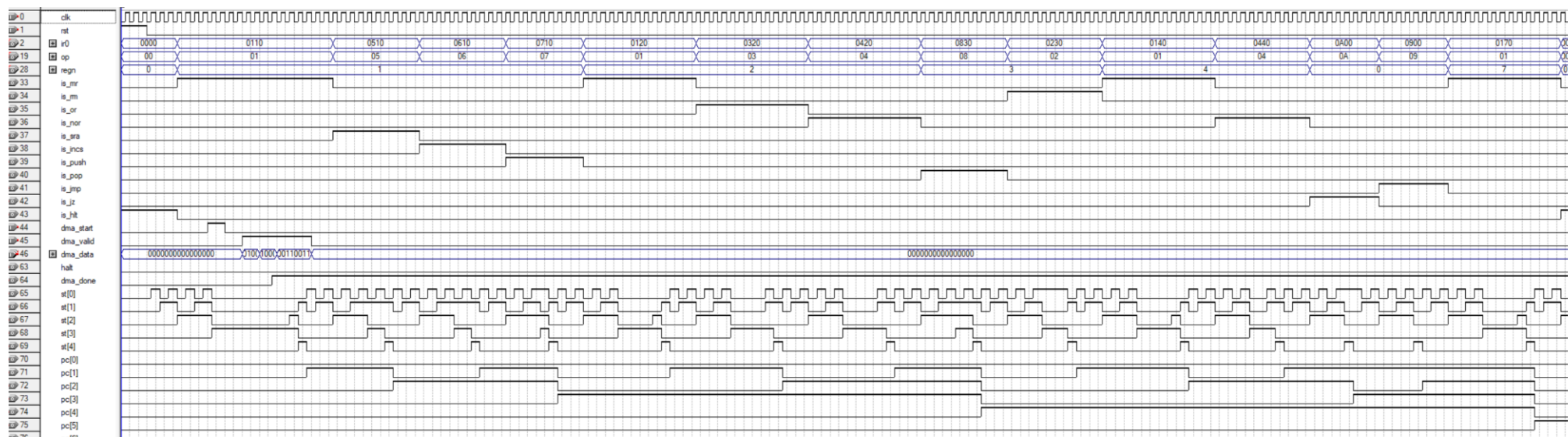


Рисунок 3.2 - Результаты функционального моделирования декодера команд

T0. При IR0 = 0110h активируется признак команды MOV adr, Rn. Это означает, что после декодирования должен быть выбран маршрут чтения из памяти данных и записи в регистр R1.

T1-T3. При IR0 = 0510h, 0610h и 0710h последовательно распознаются команды SRA R1, INCS R1 и PUSH R1. Для первых двух команд активируется исполнительный маршрут АЛУ, а для PUSH - маршрут записи в стек.

T4-T9. При IR0 = 0320h, 0420h, 0830h, 0A00h, 0900h и 0000h распознаются команды OR, NOR, POP, JZ, JMP и HLT. Каждая из них активирует только свой маршрут: чтение операнда из памяти, работу стека, загрузку PC или переход в состояние останова.

3.3 Функциональное моделирование счётчика фаз команд

Результаты функционального моделирования счётчика фаз команд показаны на рисунке 3.3. В проектируемой микро-ЭВМ команда имеет двухсловный формат, поэтому базовая часть цикла одинакова для всех операций: выборка IR0, выборка IR1, декодирование, затем переход к исполнительному состоянию.

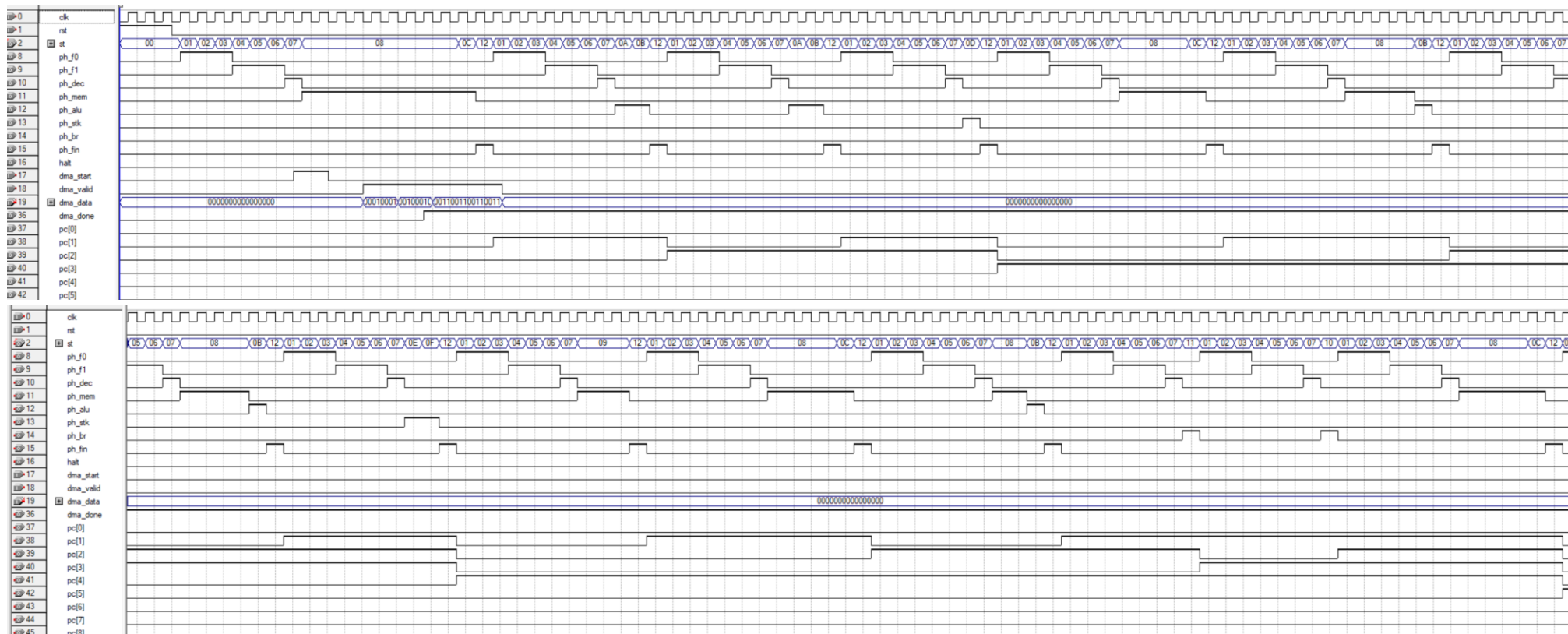


Рисунок 3.3 - Результаты функционального моделирования счётчика фаз команд

T0. Автомат находится в состоянии FETCH0_REQ: значение PC выдаётся на адрес ПЗУ, начинается запрос первого слова команды.

T1. В состоянии FETCH0_WAIT выполняется ожидание синхронного выхода ПЗУ. Затем в FETCH0_LATCH считанное слово фиксируется в регистре IR0.

T2. В состоянии FETCH1_REQ на адрес ПЗУ выдаётся $PC + 1$. Это адрес второго слова команды, содержащего непосредственный адрес операнда или перехода.

T3. В FETCH1_WAIT и FETCH1_LATCH второе слово фиксируется в IR1. После этого команда полностью загружена, и устройство управления может выполнить декодирование.

T4. В состоянии DECODE определяется дальнейший маршрут: чтение или запись памяти, операция АЛУ, команда стека, переход или останов. После завершения исполнительного маршрута автомат возвращается к выборке следующей команды.

3.4 Функциональное моделирование блока IP и IR

Результаты функционального моделирования блока указателя команд IP и регистров команды IR показаны на рисунке 3.4. На диаграмме видно, что при линейном выполнении программы PC увеличивается на два слова, а при командах передачи управления загружается адрес из IR1.

T0. При PC = 0000h в IR0 загружается 0110h, а в IR1 - 0020h. После выполнения команды MOV указатель команд принимает значение 0002h.

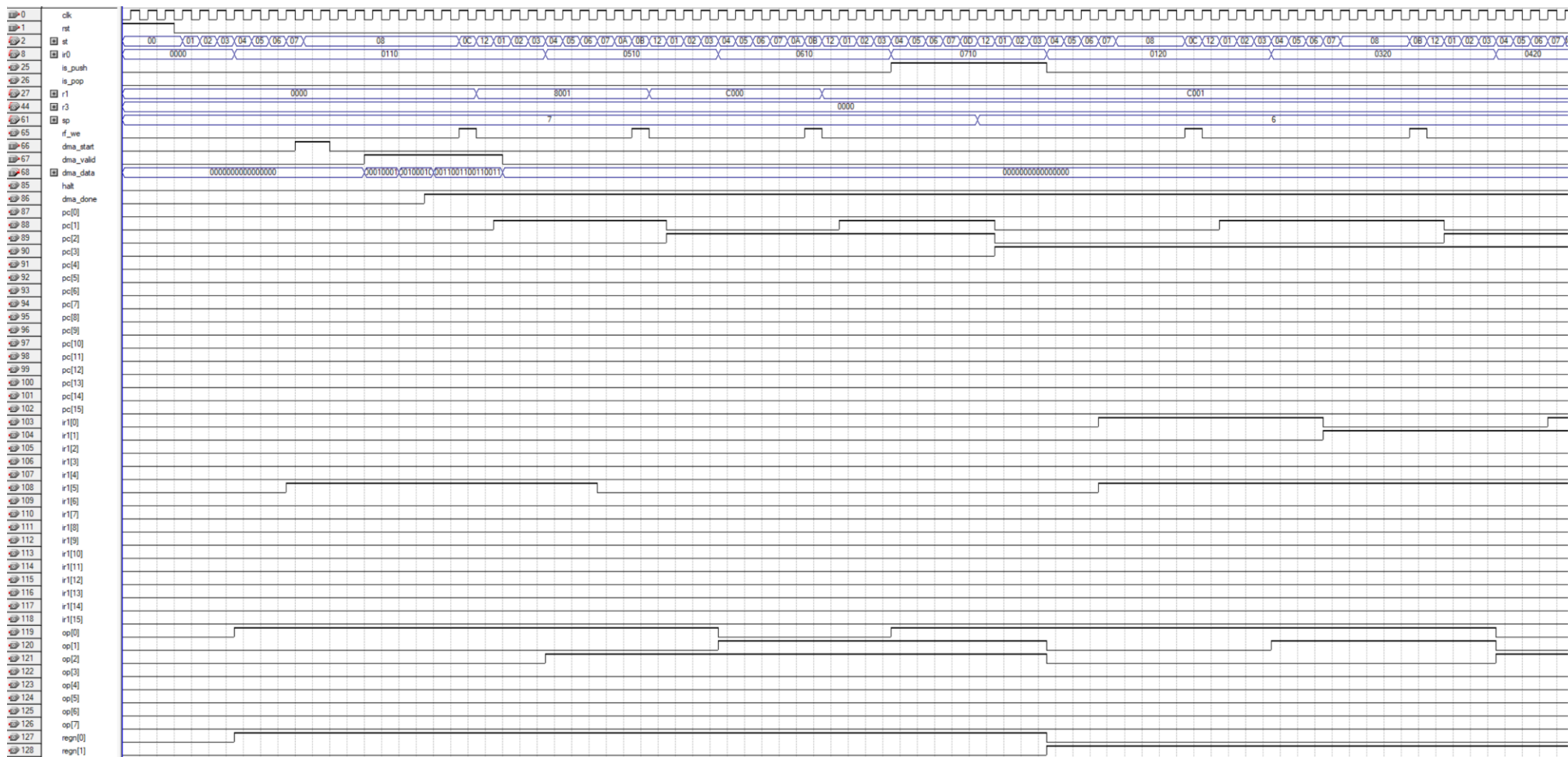
T1-T2. Для команд SRA и INCS второе слово равно 0000h и не используется как адрес данных, однако всё равно загружается в IR1 из-за единого формата команды. После исполнения PC последовательно принимает значения 0004h и 0006h.

T3. Для команды JZ в IR0 загружается 0A00h, а в IR1 - 001Ah. Так как перед этим установлен флаг Z, активируется загрузка указателя команд, и PC принимает значение 001Ah.

T4. Для команды JMP в IR1 находится 001Eh. Указатель команд загружается этим значением без проверки флагов. В конце программы при IR0 = 0000h формируется сигнал HALT.

3.5 Функциональное моделирование стекового устройства

Результаты функционального моделирования стекового устройства показаны на рисунке 3.5. Стек имеет глубину 7 слов и используется командами PUSH и POP. Работа организована по принципу LIFO: последним записанное слово считывается первым.



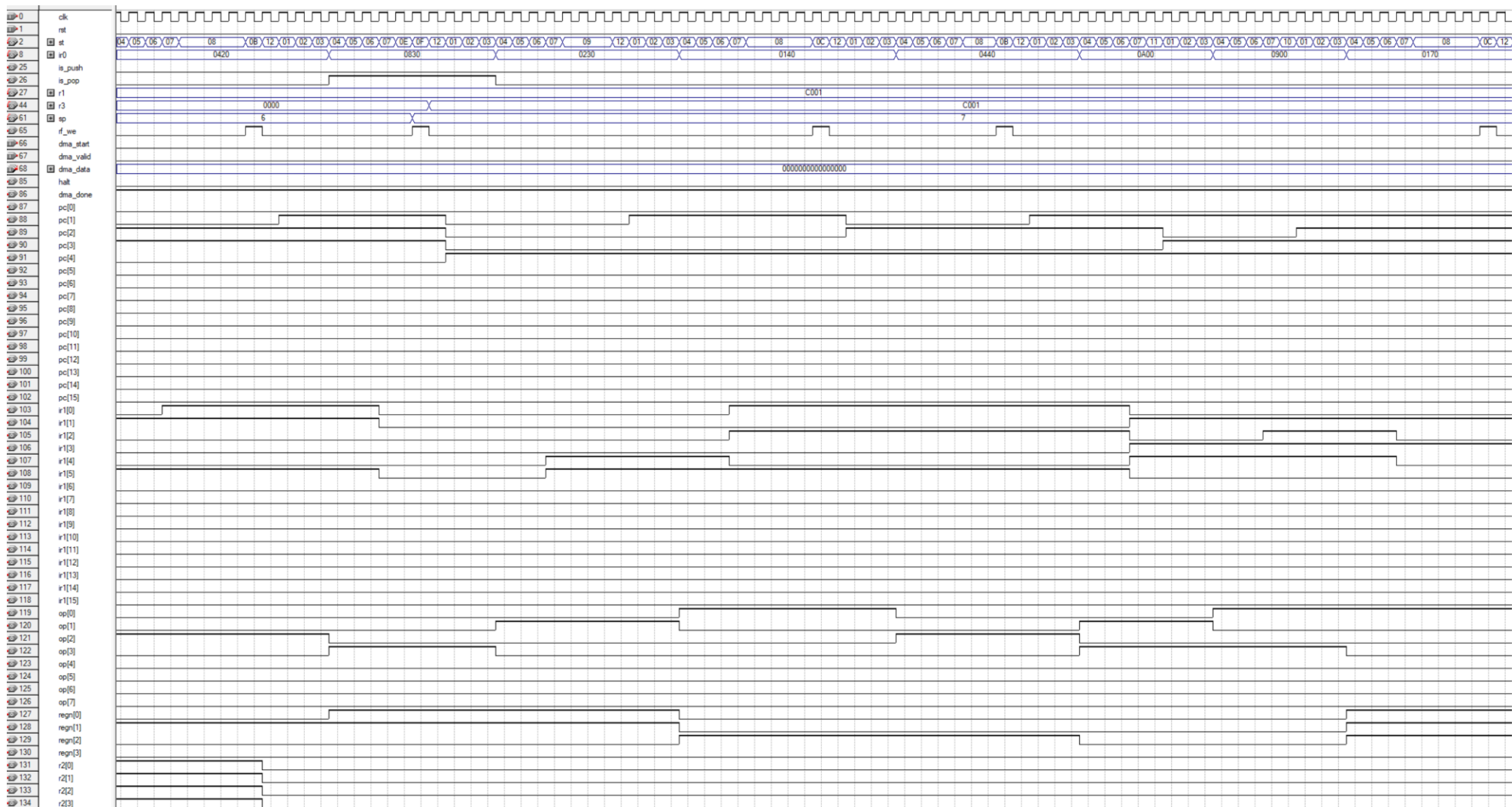


Рисунок 3.5 - Результаты функционального моделирования стекового устройства

T0. После сброса указатель стека SP находится в состоянии 7h, соответствующем пустому стеку. На диаграмме контролируется именно изменение SP и разрешение записи результата в регистровый файл.

T1. При выполнении команды PUSH R1 в стек заносится слово C001h, полученное после операций SRA и INCS. Указатель SP уменьшается с 7h до 6h, что соответствует занятию верхней ячейки стека.

T2. При выполнении команды POP R3 сохранённое слово считывается из стека и записывается в регистр R3. Указатель SP увеличивается с 6h до 7h, поэтому стек снова переходит в пустое состояние.

В полной программе стек используется парой команд PUSH R1 и POP R3. После PUSH в стек заносится значение C001h, полученное после SRA и INCS, а после POP это же значение загружается в регистр R3.

3.6 Функциональное моделирование регистров общего назначения

Результаты функционального моделирования регистров общего назначения показаны на рисунке 3.6. В проекте используется 12 регистров, а в контрольной программе задействованы R1, R2, R3, R4, R5, R6 и R7.

T0. Команда MOV 0020h, R1 загружает в R1 исходное слово 8001h. После этого операции SRA и INCS изменяют только R1, последовательно формируя C000h и C001h.

T1. Команда MOV 0021h, R2 загружает в R2 значение 00F0h. Затем операции OR и NOR используют R2 как первый операнд и записывают результат обратно в этот же регистр: 00F0h -> 0FFFh -> F000h.

T2. После команды POP R3 в регистр R3 попадает слово C001h, ранее сохранённое в стеке. Далее это слово записывается в память по адресу 0030h командой MOV R3, adr.

T3. Регистр R4 используется для проверки условного перехода: сначала в него загружается FFFFh, затем команда NOR R4, 0025h формирует 0000h и устанавливает флаг Z. Регистры R5 и R6 остаются равными нулю, поскольку соответствующие команды пропускаются переходами JZ и JMP. Регистр R7 в конце программы получает значение 1111h из области обмена КППД.

3.7 Функциональное моделирование арифметико-логического устройства

Результаты функционального моделирования арифметико-логического устройства показаны на рисунке 3.7. Проверяются операции, заданные индивидуальным вариантом: OR, NOR, SRA и INCS. На диаграмме контролируются код текущей команды, признаки выбранной операции, изменение регистров R1, R2, R4 и состояние регистра флагов.

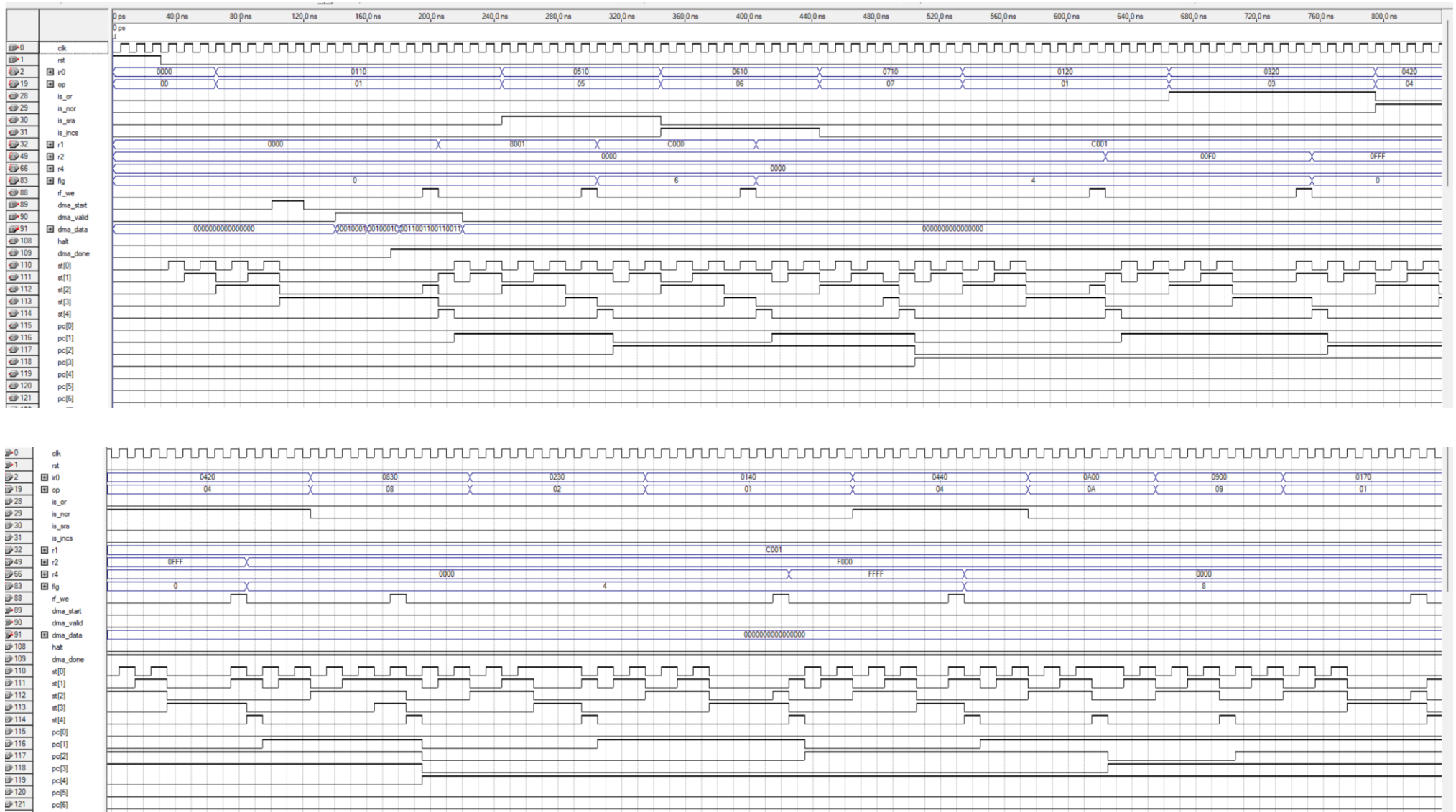


Рисунок 3.7 - Результаты функционального моделирования арифметико-логического устройства

T0. Для операции SRA на вход A подаётся 8001h. Арифметический сдвиг вправо сохраняет старший знаковый разряд, поэтому на выходе формируется C000h. Младший вытолкнутый бит попадает во флаг C, а флаг S устанавливается, так как старший бит результата равен единице.

T1. Для операции INCS используется значение C000h и установленный флаг S. При активном признаке отрицательного результата выполняется инкремент, поэтому выходное значение становится C001h.

T2. Для операции OR используются операнды 00F0h и 0F0Fh. Результат равен 0FFFh, флаг Z не устанавливается, так как результат ненулевой.

T3. Для операции NOR используются операнды 0FFFh и 0FFFh. Сначала выполняется логическое ИЛИ, затем инверсия результата, поэтому получается F000h. При проверке условного перехода дополнительно выполняется NOR над FFFFh и FFFFh, что даёт 0000h и устанавливает флаг Z.

3.8 Функциональное моделирование кэш-памяти

Результаты функционального моделирования кэш-памяти показаны на рисунке 3.8. Кэш-память обслуживает обращения процессорного ядра к ОЗУ данных. На первом обращении к новому адресу фиксируется промах, после чего формируется запрос к общей шине данных. Повторное обращение к уже загруженной строке завершается попаданием.

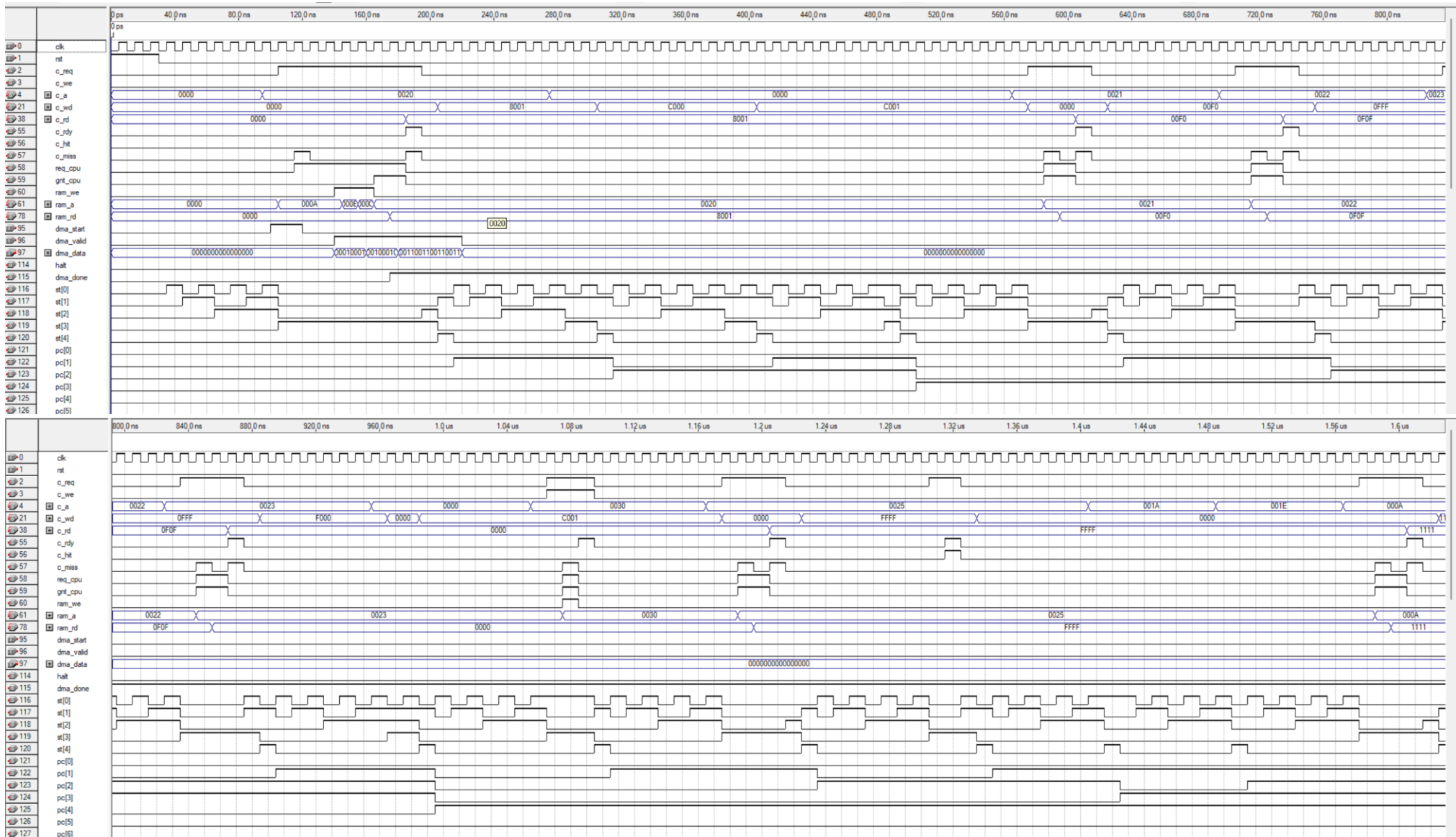


Рисунок 3.8 - Результаты функционального моделирования кэш-памяти

T0. При чтении адреса 0020h кэш ещё не содержит требуемой строки, поэтому формируется MISS и запрос REQ_CPU к арбитру шины. После получения GNT_CPU данные считываются из ОЗУ и передаются в процессор.

T1-T2. Аналогично выполняются чтения адресов 0021h и 0022h для команд MOV и OR. На этих участках кэш выступает посредником между процессором и ОЗУ, а признак MISS показывает необходимость обращения к основной памяти.

T3. При повторном обращении к адресу, строка которого уже находится в кэше, формируется HIT. В этом случае данные выдаются процессору без нового запроса к ОЗУ, что подтверждает корректную работу поля тега и признака действительности строки.

T4. Для операций записи используется согласование с ОЗУ: записываемое слово проходит через кэш-подсистему на общую шину данных, а RAM_WE активируется только после получения разрешения от арбитра.

3.9 Функциональное моделирование предсказателя переходов

Результаты функционального моделирования предсказателя переходов показаны на рисунке 3.9. В контрольной программе проверяется условный переход JZ 001Ah, выполняемый после операции NOR R4, 0025h. Эта операция формирует нулевой результат, поэтому флаг Z становится равным единице.

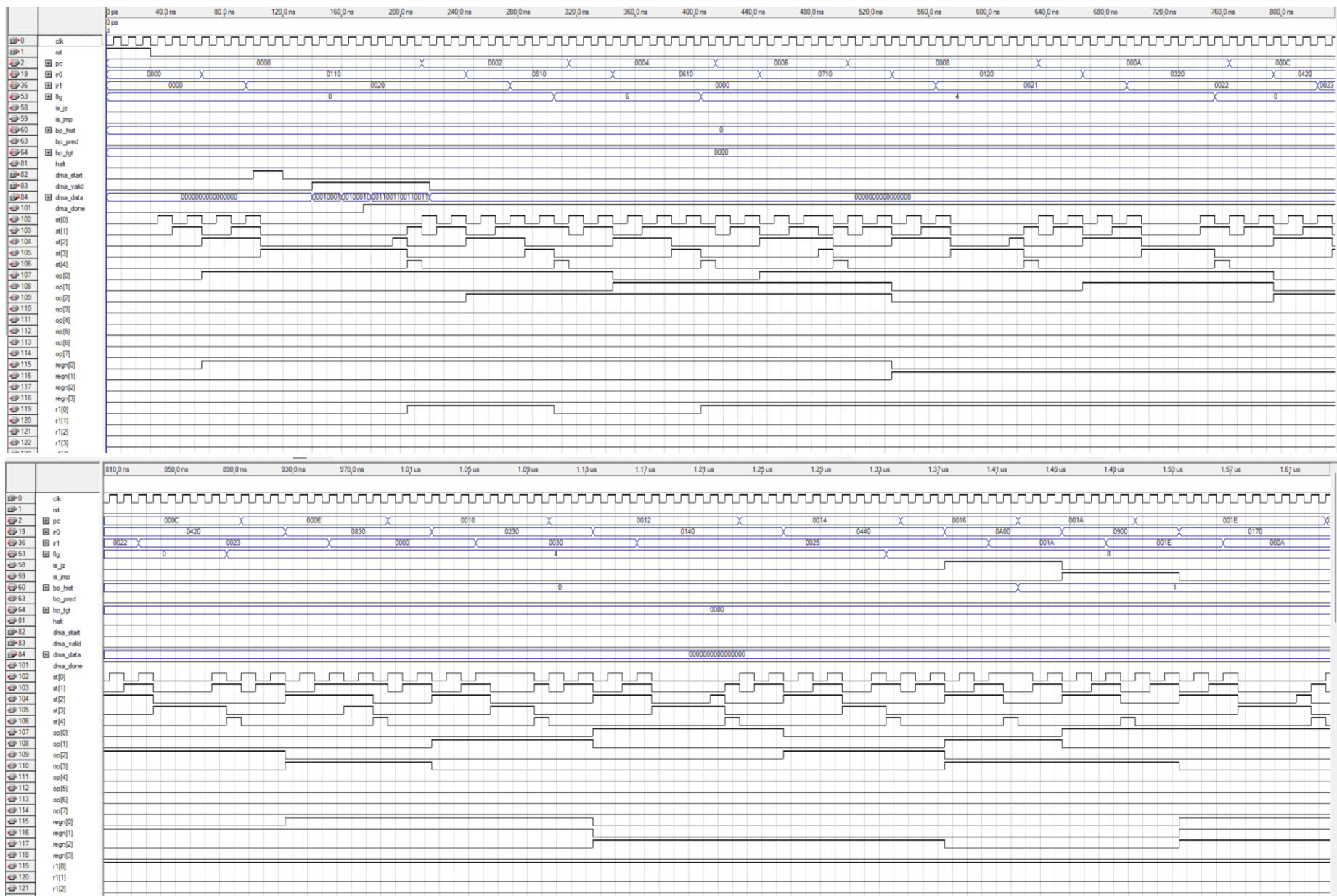


Рисунок 3.9 - Результаты функционального моделирования предсказателя переходов

Т0. До команды JZ выполняется NOR, в результате которой регистр R4 принимает значение 0000h, а в регистре флагов устанавливается Z. Это является условием выполнения перехода.

Т1. При IR0 = 0A00h и IR1 = 001Ah устройство управления переходит к обработке условного перехода. Так как в регистре флагов установлен признак Z, значение PC заменяется адресом 001Ah.

Т2. После выполнения условного перехода обновляется история предсказателя BP_HIST. Изменение этого сигнала показывает, что фактический исход перехода был передан в блок предсказания.

Т3. Следующая команда JMP 001Eh является безусловной. Для неё направление не требуется предсказывать: адрес из IR1 напрямую загружается в PC, что видно по скачку указателя команд к 001Eh.

3.10 Функциональное моделирование контроллера прямого доступа к памяти

Результаты функционального моделирования контроллера прямого доступа к памяти показаны на рисунке 3.10. КПДП используется для записи трёх контрольных слов во внутреннюю область ОЗУ без участия исполнительного тракта процессора.

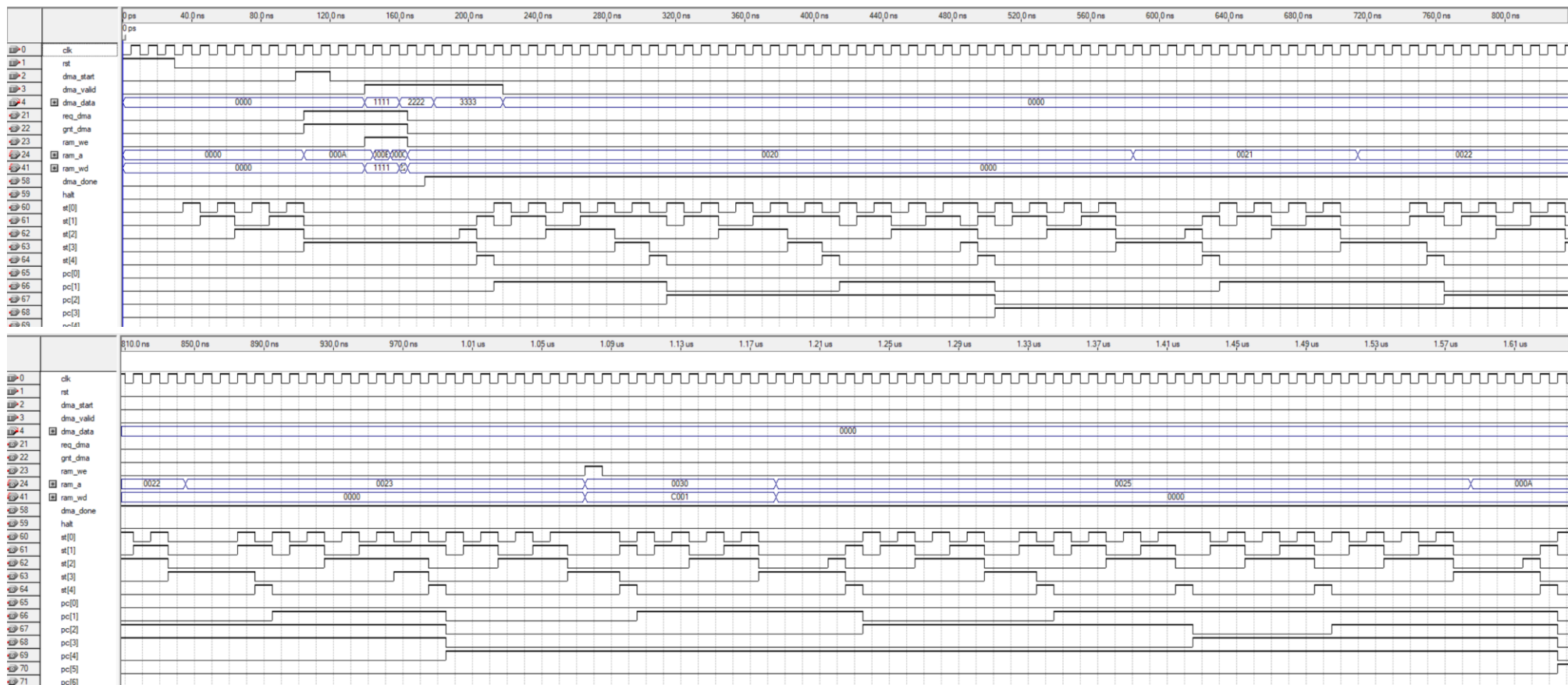


Рисунок 3.10 - Результаты функционального моделирования контроллера прямого доступа к памяти

T0. При активном dma_start контроллер переходит из состояния ожидания в состояние запроса шины. На вход dma_data подаётся первое слово 1111h, а dma_valid подтверждает его действительность.

T1. После получения разрешения gnt_dma контроллер активирует запись в ОЗУ. На адресную шину подаётся 000Ah, на шину данных - 1111h, сигнал ram_we устанавливается в единицу.

T2-T3. Следующие слова 2222h и 3333h записываются по адресам 000Bh и 000Ch. Адресный счётчик увеличивается после каждой передачи, а счётчик оставшихся слов уменьшается.

T4. После передачи последнего слова контроллер снимает запрос к шине, деактивирует RAM_WE и формирует DONE. В полной программе затем процессор считывает слово 1111h из адреса 000Ah в регистр R7, что подтверждает сохранение данных в ОЗУ.

3.11 Функциональное моделирование арбитража шин

Результаты функционального моделирования арбитража шин показаны на рисунке 3.11. Арбитр выбирает, какой мастер получает доступ к общей шине ОЗУ данных: процессорная кэш-подсистема или КПДП. В проекте приоритет имеет КПДП, так как его передача должна быть завершена без конфликтов с процессором.

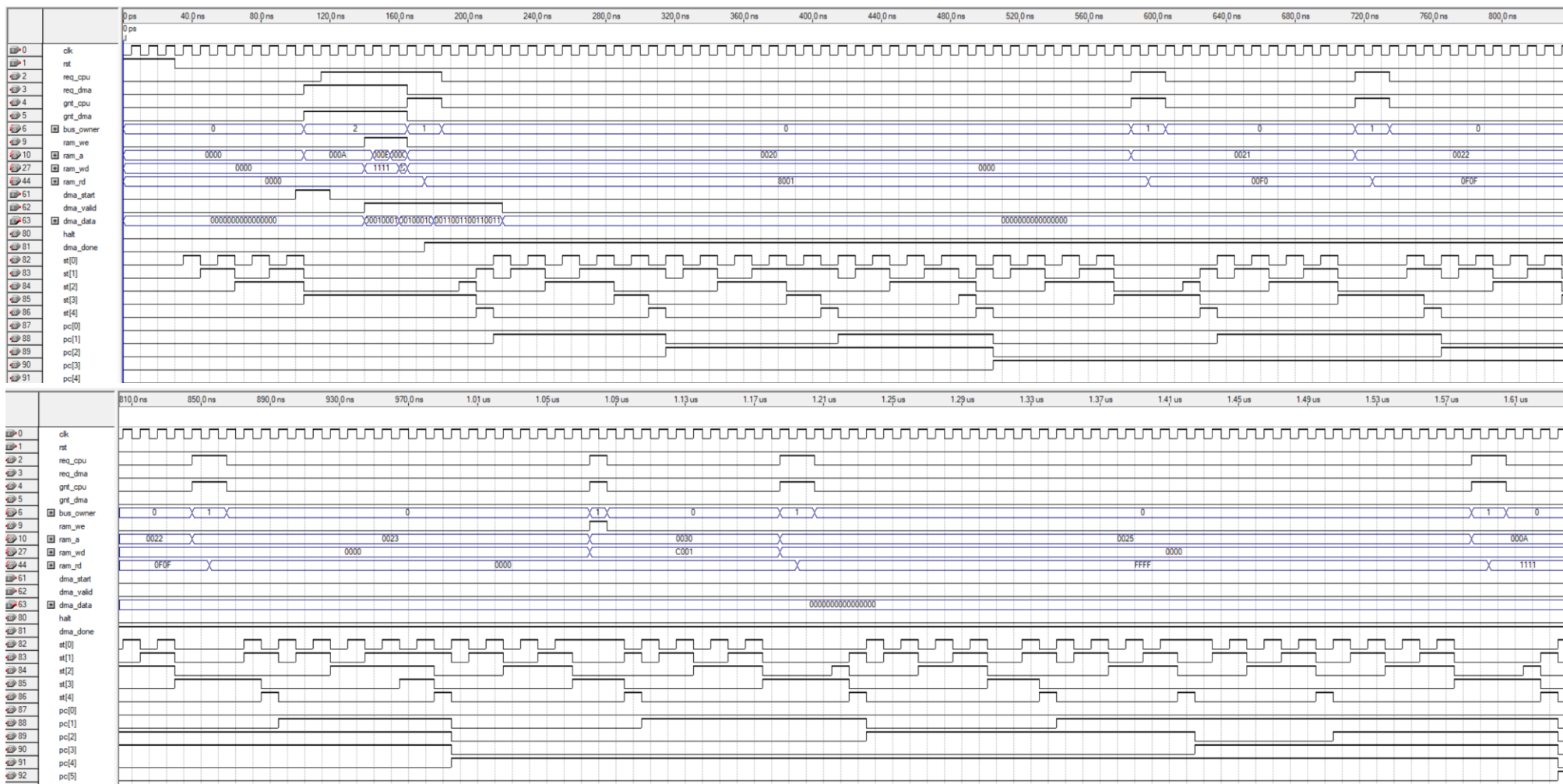


Рисунок 3.11 - Результаты функционального моделирования арбитража шин

T0. При отсутствии запросов оба разрешения GNT_DMA и GNT_CPU равны нулю, шина находится в состоянии ожидания.

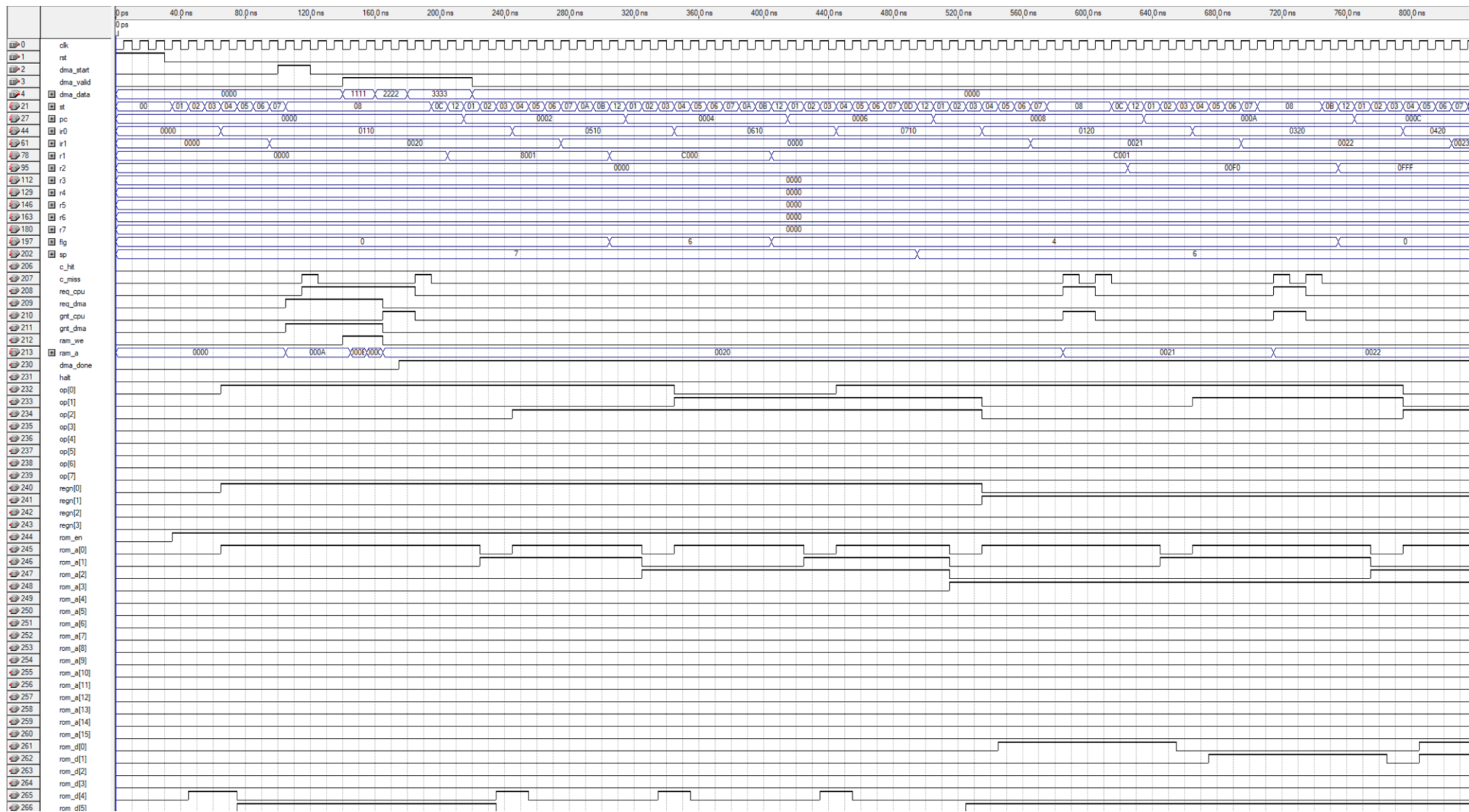
T1. При появлении REQ_DMA арбитр выдаёт GNT_DMA. Если в это же время появляется REQ_CPU, он удерживается до завершения передачи КПДП. На шине выбираются адрес, данные и признак записи от DMA-контроллера.

T2. После снятия REQ_DMA доступ может получить процессор. Если активен REQ_CPU, арбитр формирует GNT_CPU, и кэш-подсистема выполняет чтение или запись ОЗУ.

T3. При очередном обращении процессора к памяти снова активируется GNT_CPU. Отсутствие одновременной выдачи GNT_DMA и GNT_CPU подтверждает бесконфликтную работу централизованного параллельного арбитра.

3.12 Функциональное моделирование полной микро-ЭВМ

После автономной проверки блоков выполняется моделирование полной микро-ЭВМ. Результаты показаны на рисунке 3.12. На диаграмму выведены основные контрольные сигналы: такт и сброс, состояние автомата, PC, IR0, IR1, регистры R1-R7, флаги, указатель стека, запросы и разрешения шины, адреса ОЗУ, признаки предсказателя и сигнал останова.



T0. После сброса PC равен 0000h, IR0 и IR1 очищены, регистры общего назначения находятся в нулевом состоянии, HALT равен нулю. Затем начинается выборка первой команды из ПЗУ.

T1. Команда MOV 0020h, R1 загружает в R1 слово 8001h. На диаграмме это видно по появлению IR0 = 0110h, IR1 = 0020h, обращению к адресу 0020h и изменению R1 с 0000h на 8001h.

T2. Команда SRA R1 преобразует 8001h в C000h. После неё устанавливаются признаки отрицательного результата и переноса, что на диаграмме соответствует изменению регистра FLAGS.

T3. Команда INCS R1 увеличивает отрицательное значение C000h на единицу, и в R1 появляется C001h. Это слово затем используется как проверочный результат для стека.

T4. Команда PUSH R1 записывает C001h в стек, указатель SP изменяется с 7h на 6h. Далее команда MOV 0021h, R2 загружает в R2 значение 00F0h.

T5. Команда OR R2, 0022h считывает второй операнд из памяти и формирует результат 0FFFh. Следующая команда NOR R2, 0023h формирует F000h. На диаграмме это видно по последовательному изменению регистра R2.

T6. Команда POP R3 извлекает из стека сохранённое значение C001h, после чего SP возвращается к 7h, а R3 принимает значение C001h. Команда MOV R3, 0030h записывает это слово в ОЗУ по адресу 0030h; на диаграмме появляется импульс RAM_WE.

T7. Команда MOV 0025h, R4 загружает в R4 значение FFFFh. Затем NOR R4, 0025h формирует нулевой результат 0000h, поэтому устанавливается флаг Z. Это подготавливает выполнение условного перехода.

T8. Команда JZ 001Ah проверяет флаг Z. Поскольку он установлен, PC загружается адресом 001Ah, а команда MOV 0024h, R5 по адресу 0018h не выполняется. Поэтому R5 остаётся равным 0000h.

T9. Команда JMP 001Eh безусловно загружает PC значением 001Eh. Команда MOV 0024h, R6 по адресу 001Ch также пропускается, поэтому R6 сохраняет нулевое значение.

T10. Команда MOV 000Ah, R7 считывает из области обмена КППД слово 1111h и записывает его в R7. Это подтверждает совместную работу ОЗУ, КППД, арбитра и процессора.

T11. Последней выбирается команда HLT с IR0 = 0000h. Устройство управления переходит в состояние останова, сигнал HALT устанавливается в единицу, дальнейшее изменение PC прекращается.

ЗАКЛЮЧЕНИЕ

В ходе работы была разработана структура микро-ЭВМ на ПЛИС в соответствии с индивидуальным техническим заданием. Для проектируемой системы были определены и обоснованы основные архитектурные решения: Гарвардская организация памяти, 16-разрядные шины адреса и данных, синхронные ПЗУ и ОЗУ, 12 регистров общего назначения, стек глубиной 7 ячеек, кэш-память данных, блок предсказания переходов, контроллер прямого доступа к памяти и централизованный параллельный арбитр шины.

Сформирована система команд, включающая обязательные операции обмена с памятью, передачи управления и работы со стеком, а также заданные индивидуальным вариантом операции INCS, OR, NOR и SRA. Для основных функциональных блоков описаны назначение, внутренняя структура, используемые сигналы и порядок взаимодействия при выполнении программы. Особое внимание уделено маршрутизации данных между процессорным ядром, кэшем, ОЗУ и контроллером прямого доступа к памяти, а также формированию управляющих сигналов при выполнении команд различных типов. По результатам моделирования можно сделать вывод, что разработанная микро-ЭВМ выполняет операции обмена с памятью, арифметико-логические операции, работу со стеком, условный и безусловный переходы, обмен через КПДП и корректное завершение программы. Отдельные временные диаграммы блоков согласуются с итоговой диаграммой полной системы. В результате проектирования получена функционально завершённая модель микро-ЭВМ, включающая тракт выборки и декодирования команд, блок выполнения арифметико-логических операций, регистровый файл, стек, подсистему памяти и средства управления передачей данных. Работоспособность разработанной структуры подтверждается моделированием выполнения команд, включая операции обмена с памятью, стековые команды, арифметико-логические операции, команды перехода и завершение программы командой HLT.

Разработанная архитектура может рассматриваться как функционально завершённая базовая модель, пригодная для моделирования, анализа принципов работы процессорного устройства и дальнейшего развития на последующих этапах проектирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Patterson D.A., Hennessy J.L. Computer Organization and Design [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.elsevier.com/books/computer-organization-and-design/patterson/978-0-12-812275-4> – Дата доступа: 07.01.2026
- [2] Самаль Д.И. Структурная и функциональная организация ЭВМ [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://elib.bsu.by/handle/123456789/123456> – Дата доступа: 22.01.2026
- [3] Зотов В.Ю. Проектирование встраиваемых микропроцессорных систем на основе ПЛИС фирмы XILINX [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.xilinx.com/support/documentation/books/xupv2p_zotov.pdf – Дата доступа: 06.02.2026
- [4] Сергиенко А.М. Проектирование вычислительных устройств [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.solon-press.ru/books/> – Дата доступа: 06.03.2026
- [5] Поляков А.К. Языки описания цифровой аппаратуры в проектировании цифровых устройств [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.solon-press.ru/books/> – Дата доступа: 08.03.2026
- [6] Глецевич И.И., Прытков В.А., Отвагин А.В. Методические указания по дипломному проектированию для студентов специальности 40 02 01 «Вычислительные машины, системы и сети» [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://elib.bsu.by/handle/123456789/method-diplom> – Дата доступа: 12.03.2026
- [7] Intel Corporation. Documentation for FPGA Development Software [Электронный ресурс]. – Электронные данные. – Режим доступа: <http://www.altera.com/literature/> – Дата доступа: 26.03.2026
- [8] Столлингс У. Структурная организация и архитектура компьютерных систем [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.williamspublishing.com/Books/978-5-8459-0127-4.html> – Дата доступа: 25.03.2026
- [9] Таненбаум Э. Архитектура компьютерных систем [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.piter.com/collection/all/product/arhitektura-kompyuternyh-sistem-4-e-mezhdunarodnoe-izdanie> – Дата доступа: 29.03.2026
- [10] Цилькер Б.Я., Орлов С.А. Организация ЭВМ и систем [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.piter.com/collection/all/product/organizaciya-evm-i-sistem> – Дата доступа: 29.03.2026

ПРИЛОЖЕНИЕ А (обязательное)
Схема комбинированная структурная блока памяти

ПРИЛОЖЕНИЕ Б (обязательное)

Схема комбинированная структурная блока декодера команд

ПРИЛОЖЕНИЕ В (обязательное)

Схема комбинированная структурная блока счётчика фаз команд

ПРИЛОЖЕНИЕ Г (обязательное)

**Схема комбинированная структурная блока указателя команд и
регистра кода команд**

ПРИЛОЖЕНИЕ Д (обязательное)
Схема комбинированная структурная блока стекового устройства

ПРИЛОЖЕНИЕ Е (обязательное)
Схема комбинированная структурная блока регистров общего назначения

ПРИЛОЖЕНИЕ Ж (обязательное)
Схема комбинированная структурная блока арифметико-логического
устройства

ПРИЛОЖЕНИЕ И (обязательное)
Схема комбинированная структурная блока кэш-памяти

ПРИЛОЖЕНИЕ К (обязательное)

Схема комбинированная структурная блока предсказателя переходов

ПРИЛОЖЕНИЕ Л (обязательное)
Схема комбинированная структурная блока контроллера прямого
доступа к памяти

ПРИЛОЖЕНИЕ М (обязательное)
Схема комбинированная структурная блока арбитража шин

ПРИЛОЖЕНИЕ Н (обязательное)
Схема комбинированная структурная блока микро-ЭВМ

ПРИЛОЖЕНИЕ П (обязательное)
Листинг проекта